Senior Android / Kotlin Interview Study Guide

Study guide, question bank, mock interviews, flashcards, and references.

Generated from the Markdown study files in the interview-guide folder.

Table of Contents

Question Coverage **164**

References **231**

Study Guide

> *Quality status: **90/100, Student-Facing Draft**. Target: **98+**. Main gap: keep expanding source-backed question variants, add more practice blocks, and connect every question-bank item to a strong answer.*

This is the student-facing guide. Read this first.

The structure is:

- Theory you need to understand.
- Interview questions they may ask.
- Strong answer you can practice out loud.
- Follow-ups they may use to go deeper, with answers directly underneath.
- What not to say.

Internal research files such as `15-failure-analysis.md`, `16-quality-upgrade-plan.md`, and `17-chapter-scorecard.md` are not study chapters. They exist to improve this guide.

How To Use This Guide

Study in this order:

- Read the theory for one topic.
- Answer the interview questions out loud before reading the strong answer.
- Compare your answer against the strong answer.
- Practice the follow-ups.
- Drill the same topic in `QUESTION-BANK.md`.
- Use `FLASHCARDS.md` for quick daily review.
- Use `MOCK-INTERVIEWS.md` once you can answer without reading.

Do not memorize the answers word-for-word. The goal is to sound like you understand the topic, not like you are reciting a page.

How To Study One Topic

Use this loop for every topic:

- Read the theory until you can explain it without looking.
- Cover the strong answer and answer the interview question out loud.
- Compare your answer with the strong answer.
- Answer every follow-up out loud, then compare against the written follow-up answer.
- If your answer sounds like a definition, rewrite it as a practical explanation.
- Drill 5-10 related questions from `QUESTION-BANK.md`.
- Revisit the flashcards the next day.

Good interview answers usually do three things at the same time:

- explain the concept,
- show how it behaves in Android/Kotlin,
- mention the trap that catches weaker candidates.

Study Roadmap

Two-Week Fast Track

Week 1:

- Day 1: Kotlin data classes, equality, null safety.
- Day 2: Kotlin generics, sealed classes, inline/reified.
- Day 3: Android lifecycle, ViewModel, process death.
- Day 4: Coroutines basics, scope, dispatchers, cancellation.
- Day 5: Flow, StateFlow, SharedFlow, lifecycle collection.
- Day 6: Compose state, recomposition, effects.
- Day 7: Review flashcards and run mock rounds 1-4.

Week 2:

- Day 8: MVVM, MVI, Clean Architecture.
- Day 9: Repository, UseCases, UDF, modularization.
- Day 10: Mobile system design: offline-first, sync, WorkManager.
- Day 11: Testing: ViewModel, Flow, Compose, Room migrations.
- Day 12: Performance, security, release builds.
- Day 13: Soft skills and project deep dives.
- Day 14: Mixed mock interview and weak-area review.

Four-Week Complete Track

- Week 1: Kotlin and Android fundamentals.
- Week 2: Coroutines, Flow, Compose.
- Week 3: Architecture, design patterns, mobile system design.
- Week 4: Testing, performance, security, release, soft skills, mock interviews.

Interview Round Map

Most Senior Android/Kotlin interview loops can include:

- Kotlin fundamentals round.
- Android fundamentals/lifecycle round.
- Coroutines/Flow/Compose round.
- Architecture round.
- Mobile system design round.
- Practical coding or debugging round.

- Testing/performance round.
- Behavioral/leadership round.

Your answers should connect fundamentals to production Android behavior: lifecycle, process death, performance, background limits, testing, and trade-offs.

Answering Framework

For conceptual questions:

- Answer directly.
- Explain in practical terms.
- Add Android/Kotlin implication.
- Mention a trap or trade-off.
- Close with how you use it in practice.

For system design:

- Clarify requirements.
- Define source of truth.
- Explain data flow.
- Explain offline/background behavior.
- Explain errors, retries, conflicts.
- Explain testing and rollout.
- Call out trade-offs.

For behavioral questions:

- Context.
- Constraint.
- Your action.
- Trade-off.
- Result.
- Reflection.

---

1. Kotlin Fundamentals

Documentation Anchors

- Kotlin data classes (https://kotlinlang.org/docs/data-classes.html)
- Kotlin null safety (https://kotlinlang.org/docs/null-safety.html)
- Kotlin equality (https://kotlinlang.org/docs/equality.html)
- Kotlin sealed classes and interfaces (https://kotlinlang.org/docs/sealed-classes.html)
- Kotlin generics (https://kotlinlang.org/docs/generics.html)

- Kotlin inline functions (https://kotlinlang.org/docs/inline-functions.html)

Theory To Know

For Senior Android interviews, Kotlin fundamentals are not just syntax. Interviewers often start with simple Kotlin questions, then push into behavior: what the compiler generates, what happens at runtime, how Java interop changes safety, and how language features affect architecture.

The goal is not to define terms. The goal is to explain how Kotlin helps model state safely, where it can still fail, and what trade-offs matter in Android codebases.

Important concepts:

- `val` vs `var`
- nullable vs non-nullable types
- platform types from Java
- `data class` generated methods
- `==` vs `===`
- `equals` / `hashCode` contract
- shallow `copy()`
- sealed classes/interfaces
- object and companion object
- generics: `in`, `out`, type erasure
- `inline` and `reified`

Interview Question: What is a Kotlin data class?

Asked As / Variations

- What is a data class?
- What methods does a data class generate?
- How does equality work in a data class?
- Why can a data class be dangerous as a `HashMap` key?
- What happens if a data class has mutable properties?

Strong Answer

"A Kotlin data class is useful when a class mainly represents a value, state, DTO, or simple domain model. The compiler generates `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions based on the properties in the primary constructor.

The important part is that equality is structural when using `==`. That means Kotlin calls `equals` and compares the primary constructor properties. It does not compare object identity unless I use `===`.

I am also careful with mutability. `copy()` is shallow, so nested mutable objects are shared unless I explicitly copy them. And if a data class is used as a key in a `HashMap` or stored in a `HashSet`, properties involved in equality and hash code should not change after insertion. Otherwise the collection may not find the object later."

Tricky Follow-Up Questions And Answers

Follow-up: What methods does a data class generate?

Answer: "For primary constructor properties, Kotlin generates `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions. `componentN` is what enables destructuring. The key detail is primary constructor properties: fields declared inside the class body are not part of those generated methods."

Follow-up: Are properties inside the class body included in equals?

Answer: "No. Generated `equals` and `hashCode` only use properties declared in the primary constructor. A property declared inside the class body can still exist and change, but it will not affect generated equality. That matters because two instances can compare equal even if a body property has a different value."

Follow-up: How does generated equals work internally?

Answer: "Conceptually it first checks whether both references are the same object, then checks whether the other value is the same data class type, then compares each primary-constructor property in order using equality for that property. For a data class with five primary-constructor properties, the generated equality path can compare up to five properties after the cheap identity/type checks. It may stop earlier as soon as one property is different. That detail matters in interviews because it shows I know `==` is structural for data classes, but it is still implemented as ordinary equality checks over the properties that define the value."

Follow-up: Does copy() do a deep copy?

Answer: "No. `copy()` is shallow. It creates a new outer object, but if a property points to a mutable list or another mutable object, both instances can still share that same nested object. In UI state, I usually prefer immutable nested data or explicit nested copies."

Follow-up: When is hashCode used?

Answer: "`hashCode` is used by hash-based collections like `HashMap` and `HashSet`. The collection uses the hash to narrow lookup, then `equals` to confirm the match. Equal objects must have the same hash code. If a property used in `hashCode` changes after insertion, lookups can break."

What Not To Say

"A data class is just a class that stores data."

Interview Question: Kotlin is null-safe. Can it still throw NPE?

Asked As / Variations

- If Kotlin is null-safe, why do we still see crashes from null?
- What is a platform type?
- How do you handle Java APIs from Kotlin?
- When do you use `!!`?
- How do you model absence in UI state?

Strong Answer

"Yes. Kotlin reduces null pointer errors by making nullability explicit in the type system, but it does not make NPEs impossible. A `String` is treated as non-null, while `String?` forces me to handle absence. But NPEs can still come from `!!`, Java interop, platform types, bad initialization, `lateinit`, explicit throws, or generic edge cases.

In Android this matters because many APIs still cross Java boundaries. Around those boundaries, I try to normalize the value early: check it, map it, or wrap it in a safer API. I also avoid using null to represent too many states. For UI, a sealed state like `Loading`, `Content`, and `Error` is often clearer than multiple nullable fields."

Tricky Follow-Up Questions And Answers

Follow-up: What is a platform type?

Answer: "A platform type comes from Java interop when Kotlin cannot know whether the value is nullable. Kotlin lets me treat it as nullable or non-null, which is convenient but risky. Around Java or Android framework boundaries, I prefer to normalize the value early instead of letting uncertainty spread through the app."

Follow-up: When is !! acceptable?

Answer: "Very rarely. I would use `!!` only when the invariant is guaranteed but the compiler cannot prove it, and I want a fail-fast crash if the invariant is broken. In most production code, I prefer a null check, `requireNotNull`, safer modeling, or moving the value into a non-null state earlier."

Follow-up: lateinit vs nullable property?

Answer: "`lateinit` means I promise the value will be initialized before use, and the app will crash if that promise is wrong. A nullable property means absence is a valid state that callers must handle. I use `lateinit` only when lifecycle or injection guarantees are clear; otherwise nullable state or constructor injection is safer."

Follow-up: Empty list or null list?

Answer: "If the request succeeded and there are no items, I return an empty list. If data is not loaded, failed, or unavailable, that should be modeled separately. A null list often mixes too many states into one value."

Interview Question: Sealed class vs enum?

Asked As / Variations

- How would you model loading, success, and error?
- Why not use several booleans for UI state?
- Enum or sealed class for API result?
- How do you make state handling exhaustive?

Strong Answer

"I use an enum when I have a fixed set of constants that all have the same shape, like sort order or a simple status. I use a sealed class or sealed interface when each case may carry different data or behavior.

For Android UI state, sealed types are often better because they make invalid combinations harder to represent. Instead of having `isLoading`, `data`, and `error` all nullable or conflicting, I can model `Loading`, `Content(data)`, and `Error(message)` as separate cases. Then a `when` expression can force me to handle all cases.

The trap is overusing sealed classes for simple constants. If every case is just a name and no case carries data, enum may be simpler. The point is to model the domain clearly, not to use the most advanced language feature."

Tricky Follow-Up Questions And Answers

Follow-up: Can sealed classes help with UI state?

Answer: "Yes. They make mutually exclusive states explicit. That is useful when only one of loading, content, empty, or error should exist at a time."

Follow-up: Sealed class or sealed interface?

Answer: "A sealed class can hold state and constructor logic. A sealed interface is useful when different classes need to implement a restricted hierarchy, especially if they already extend something else."

Interview Question: Why does `reified` require `inline`?

Asked As / Variations

- What is type erasure?
- Why can some Kotlin functions access `T::class`?
- Why do JSON helpers use `inline reified`?
- Explain `in` and `out` generics.

Strong Answer

"On the JVM, generic type information is erased at runtime, so inside a normal generic function I cannot reliably ask for the actual type `T`. Kotlin's `reified` type parameters work only in inline functions because the compiler copies the function body to the call site and can substitute the real type there.

That is why APIs like `fromJson<T>()`, navigation helpers, or type-safe service lookups often use `inline reified`. It makes the call site clean because I do not have to pass `Class<T>` manually.

The trade-off is that inline functions increase bytecode at call sites and should not be used just because they look clever. I use `reified` when runtime type access makes the API safer or more readable."

Tricky Follow-Up Questions And Answers

Follow-up: What is type erasure?

Answer: "Type erasure means generic type arguments are mostly not available at runtime on the JVM. A `List<String>` and `List<Int>` are both just `List` at runtime in many contexts."

Follow-up: What are in and out?

Answer: "`out T` is for producers: safe to return `T`. `in T` is for consumers: safe to accept `T`. The shortcut is producer out, consumer in."

Interview Question: Which Kotlin features matter most in Android architecture?

Asked As / Variations

- What are scope functions and when can they hurt readability?
- What is a value class?
- When would you use `Result`, sealed classes, or exceptions?
- Can extension functions override member functions?
- Companion object vs object?

Strong Answer

"I would not answer this as a list of clever Kotlin features. I would connect each feature to modeling and maintainability.

Data classes are good for value-like state, but I watch mutability and equality. Sealed classes or sealed interfaces help model a closed set of states, especially UI states and API results. Value classes can make domain primitives more explicit, like `UserId` instead of passing raw `String`, with less allocation overhead in many cases, but they still have constraints and should not be used just for style.

Scope functions like `let`, `run`, `apply`, `also`, and `with` are useful when they make object construction or null handling clearer. They become a problem when nested chains hide the receiver or make side effects hard to follow. Extension functions are statically resolved helpers; they do not override member functions. For shared behavior, I still need real polymorphism, interfaces, or composition.

For errors, I use exceptions for exceptional failures, sealed result types when the caller must handle known outcomes, and Kotlin `Result` carefully because it can be convenient but may hide domain meaning. The senior point is choosing the feature that makes invalid states harder to represent and code easier to reason about."

Tricky Follow-Up Questions And Answers

Follow-up: Can extension functions override member functions?

Answer: "No. Extension functions are resolved statically based on the declared receiver type. If a class has a member function with the same signature, the member wins. I use extensions for convenience at boundaries, not to simulate inheritance."

Follow-up: Companion object vs object?

Answer: "An `object` declares a singleton. A `companion object` is a singleton associated with a class and is often used for factories, constants, or JVM interop. I avoid putting business dependencies in them because that recreates global state."

Follow-up: When is a value class useful?

Answer: "A value class is useful when a primitive needs stronger domain meaning, such as `UserId`, `Email`, or `MoneyCents`. It can prevent mixing unrelated strings or numbers. I still keep it simple because serialization, Java interop, generics, and framework boundaries can add constraints."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: What is a Kotlin data class?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 2: What functions does a data class generate?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 3: Which properties are included in generated `equals` and `hashCode`?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 4: Are properties declared inside the class body included in `copy()`?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 5: What is the difference between `==` and `===`?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 6: How does generated `equals` work internally?

Senior answer: "For a data class, generated `equals` is ordinary structural equality over the primary-constructor properties. Conceptually it checks quick cases first, such as same reference and compatible type, then compares each primary-constructor property in order using that property's equality. If the class has five primary-constructor properties, it can compare up to five properties after the cheap checks, but it can stop earlier when the first property differs. Body properties are not part of that generated equality. I would also connect this to `==`: in Kotlin, `==` calls `equals`, while `===` asks whether both references point to the same object."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 7: How many comparisons can `equals` do for a data class with five properties?

Senior answer: "For a data class, generated `equals` is ordinary structural equality over the primary-constructor properties. Conceptually it checks quick cases first, such as same reference and compatible type, then compares each primary-constructor property in order using that property's equality. If the class has five primary-constructor properties, it can compare up to five properties after the cheap checks, but it can stop earlier when the first property differs. Body properties are not part of that generated equality. I would also connect this to `==`: in Kotlin, `==` calls `equals`, while `===` asks whether both references point to the same object."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 8: When is `hashCode` used?

Senior answer: "`hashCode` is used by hash-based collections such as `HashMap` and `HashSet` to narrow where an object might live. The collection uses the hash to find a bucket and then uses `equals` to confirm the actual match, because different objects can share a hash. The contract is: if two objects are equal, they must have the same hash code. The reverse is not guaranteed. For data classes, generated `hashCode` uses the same primary-constructor properties as generated `equals`, so mutating those properties after insertion into a hash collection can make the object hard or impossible to find through normal lookup."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 9: Why must `equals` and `hashCode` be consistent?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 10: What can go wrong if a mutable data class is used as a `HashMap` key?

Senior answer: "If a mutable data class is used as a `HashMap` key, the dangerous part is not the word data class; it is that the fields used by `equals` and `hashCode` can change after insertion. A hash map chooses a bucket from the original hash. If the key changes later, the object may now produce a different hash, so lookup, remove, or contains can fail even though the object is still physically inside the map. In an interview I would say: keys must be effectively immutable, or equality/hash code must be based only on stable identity. The same idea matters for `HashSet`, DiffUtil, and UI state comparisons."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 11: Is `copy()` deep or shallow?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 12: When should you avoid a data class?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 13: Kotlin is null-safe. Can it still throw `NullPointerException`?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 14: What is a platform type?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 15: When would you use `!!`?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 16: What is the difference between `String`, `String?`, and a Java platform type?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 17: What is the difference between `lateinit`, `lazy`, and nullable properties?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 18: Would you return `null` or an empty list from a repository?

Senior answer: "I would not choose `null` or empty list mechanically; I would choose based on meaning. An empty list means the request succeeded and there are currently zero items. `null` means absence, unknown, not loaded, or not applicable, and that should be explicit because it creates a different UI and data contract. In a repository, I usually avoid `null` collections unless absence is a real domain state. For loading/error/content, I prefer a typed result or UI-state model instead of overloading `null` and empty. That answer prevents bugs where an error or not-yet-loaded state is accidentally rendered as a valid empty screen."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 19: How do you model loading, empty, content, and error states?

Senior answer: "I would focus on modeling the allowed states. Kotlin gives me enums for fixed constants, sealed classes or sealed interfaces for closed alternatives that may carry different data, and typed results for success/failure flows. In Android this matters because UI often has distinct states: loading, empty, content, error, permission required, or stale cached content. A sealed model makes the `when` exhaustive and prevents impossible combinations like `isLoading=true` with stale error data unless I intentionally model that. I would still avoid over-modeling: if the values are simple constants with the same shape, an enum is easier to read and maintain."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 20: What is a sealed class?

Senior answer: "A sealed class represents a closed hierarchy: the compiler knows the permitted subtypes, so `when` expressions can be exhaustive without an `else` when every case is handled. I use it when alternatives carry different data or behavior, such as `Loading`, `Content(items)`, `Empty`, and `Error(cause)`. An enum is better for a fixed list of constants with the same shape, such as sort order or theme mode. The senior detail is choosing the model that makes impossible states impossible: sealed hierarchies are great for typed UI state and domain results, but they can be overkill for simple constant sets."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 21: Sealed class vs enum?

Senior answer: "A sealed class represents a closed hierarchy: the compiler knows the permitted subtypes, so `when` expressions can be exhaustive without an `else` when every case is handled. I use it when alternatives carry different data or behavior, such as `Loading`, `Content(items)`, `Empty`, and `Error(cause)`. An enum is better for a fixed list of constants with the same shape, such as sort order or theme mode. The senior detail is choosing the model that makes impossible states impossible: sealed hierarchies are great for typed UI state and domain results, but they can be overkill for simple constant sets."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 22: What is an object declaration?

Senior answer: "I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered:

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Question 23: Companion object vs object?

Senior answer: "I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered:

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Question 24: What are extension functions?

Senior answer: "I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered:

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Question 25: Can extension functions override member functions?

Senior answer: "I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered:

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Question 26: What are Kotlin scope functions and when can they hurt readability?

Senior answer: "I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered:

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Question 27: What does `inline` do?

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

Question 28: Why does `reified` require `inline`?

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

Question 29: What is type erasure?

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

Question 30: Explain `in` and `out` in Kotlin generics.

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

Question 31: How would you explain a data class without using the word "boilerplate"?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 32: What happens if a data class contains a mutable list?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 33: How do data classes interact with DiffUtil or Compose state comparisons?

Senior answer: "I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered:

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

Question 34: Why can `lateinit` be dangerous?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 35: When is `lazy` initialized?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 36: How would you model an API result in Kotlin?

Senior answer: "I would focus on modeling the allowed states. Kotlin gives me enums for fixed constants, sealed classes or sealed interfaces for closed alternatives that may carry different data, and typed results for success/failure flows. In Android this matters because UI often has distinct states: loading, empty, content, error, permission required, or stale cached content. A sealed model makes the `when` exhaustive and prevents impossible combinations like `isLoading=true` with stale error data unless I intentionally model that. I would still avoid over-modeling: if the values are simple constants with the same shape, an enum is easier to read and maintain."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 37: What is the difference between `Result`, sealed classes, and exceptions?

Senior answer: "I would focus on modeling the allowed states. Kotlin gives me enums for fixed constants, sealed classes or sealed interfaces for closed alternatives that may carry different data, and typed results for success/failure flows. In Android this matters because UI often has distinct states: loading, empty, content, error, permission required, or stale cached content. A sealed model makes the `when` exhaustive and prevents impossible combinations like `isLoading=true` with stale error data unless I intentionally model that. I would still avoid over-modeling: if the values are simple constants with the same shape, an enum is easier to read and maintain."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 38: Can a sealed class be extended outside its package/module?

Senior answer: "I would focus on modeling the allowed states. Kotlin gives me enums for fixed constants, sealed classes or sealed interfaces for closed alternatives that may carry different data, and typed results for success/failure flows. In Android this matters because UI often has distinct states: loading, empty, content, error, permission required, or stale cached content. A sealed model makes the `when` exhaustive and prevents impossible combinations like `isLoading=true` with stale error data unless I intentionally model that. I would still avoid over-modeling: if the values are simple constants with the same shape, an enum is easier to read and maintain."

Tricky follow-ups answered:

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

Question 39: What is a value class?

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

Question 40: When would a value class help in domain modeling?

Senior answer: "I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered:

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

2. Android Fundamentals

Documentation Anchors

- Android app architecture (https://developer.android.com/topic/architecture)
- ViewModel overview (https://developer.android.com/topic/libraries/architecture/viewmodel)
- Saved state module for ViewModel (https://developer.android.com/topic/libraries/architecture/viewmodel/viewmodel-savedstate)
- Processes and app lifecycle (https://developer.android.com/guide/components/activities/process-lifecycle)
- Intents and intent filters (https://developer.android.com/guide/components/intents-filters)
- App permissions (https://developer.android.com/training/permissions/requesting)

Theory To Know

Senior Android interviews test lifecycle ownership. The interviewer is usually checking whether you understand that Android can recreate UI objects, kill the process, restore small state, and leave you with stale references if ownership is wrong.

The big mental model is lifetime. UI objects are short-lived. ViewModels live longer than a single Activity/Fragment instance but do not survive process death. Durable data must live outside memory.

Important concepts:

- Activity and Fragment lifecycle
- configuration change vs process death
- ViewModel lifetime
- `SavedStateHandle`
- saved instance state limits
- context types
- memory leaks from lifecycle references
- intents, deep links, permissions, services, broadcasts

Interview Question: What survives rotation, and what survives process death?

Asked As / Variations

- Does ViewModel survive process death?
- User rotates the phone during a request. What happens?
- The app is killed in background and restored. What state remains?
- Where do you store selected tab, form input, and cached data?

Strong Answer

"Rotation and process death are different. During rotation, the Activity is recreated, but a ViewModel scoped to that screen or navigation entry can survive. That makes ViewModel a good place for screen state and screen-owned work.

Process death is different. If the OS kills the app process, all memory is gone: Activities, Fragments, ViewModels, singletons, repositories, coroutine scopes, and cached fields. When the user returns, Android recreates components, but the old objects are not restored.

For restoration, I save small pieces of state like IDs, selected tab, filters, or simple form values with `SavedStateHandle`, `rememberSaveable`, or saved instance state. Durable data belongs in Room, DataStore, files, or the backend. The screen should be able to rebuild itself from saved keys plus durable data."

Tricky Follow-Up Questions And Answers

Follow-up: Does ViewModel survive process death?

Answer: "No. A ViewModel can survive configuration changes while its owner is still valid, but process death removes the whole app process. After process death, the ViewModel is recreated. Any important state must be restored from saved state or persistent storage."

Follow-up: Are singletons preserved after process death?

Answer: "No. Singletons are only singletons inside the current process. If the process dies, they are gone too. That is why a singleton repository is not persistence."

Follow-up: What belongs in SavedStateHandle?

Answer: "Small restoration values: IDs, selected tab, filters, simple form fields, or navigation arguments. I would not store large lists, bitmaps, repositories, or complex object graphs there."

Follow-up: What should go to Room or DataStore?

Answer: "Durable data should go outside saved state. Room fits structured relational or cached app data, especially data the UI can observe and rebuild from. DataStore fits small key-value or typed preferences. If losing it after process death would break the user experience, it should not live only in memory or a Bundle."

What Not To Say

"ViewModel survives lifecycle, so it survives process death."

Interview Question: Why should a ViewModel not hold an Activity or View reference?

Strong Answer

"A ViewModel can outlive a specific Activity or Fragment instance during configuration changes. If it holds an Activity, Fragment, View, or binding reference, it can keep the old UI instance alive after it should be destroyed. That creates memory leaks and can also cause updates to go to the wrong UI object.

UI work should stay in the UI layer. The ViewModel should expose state and events, not directly manipulate views. If I truly need context-like functionality, I prefer injecting an abstraction or using application context carefully, not holding an Activity reference."

Interview Question: Application context vs Activity context?

Asked As / Variations

- Why can context cause memory leaks?
- When should you use application context?
- Can a repository hold context?

- Why is Activity context dangerous in a singleton?

Strong Answer

"The difference is lifetime. Application context lives as long as the app process. Activity context is tied to a specific Activity instance and carries UI-related configuration like theme, window, and current resources.

If I need something process-wide, like creating a database, DataStore, or a dependency that should not depend on a screen, application context is usually safer. If I need UI-specific behavior, like inflating themed views, showing dialogs, or starting UI flows, Activity context may be required.

The trap is passing Activity context into long-lived objects like singletons, repositories, or ViewModels. That can leak the Activity after rotation. So I always ask: how long will this object live, and does it really need UI context?"

Interview Question: BroadcastReceiver vs Service vs WorkManager?

Strong Answer

"A BroadcastReceiver reacts to broadcast events and should do quick work. A Service is for longer-running work, especially when it is user-visible as a foreground service. WorkManager is for deferrable persistent work that should survive process death and can run with constraints and retry.

I choose based on the guarantee I need. If I only need to react quickly to an event, a receiver can be enough. If work must continue immediately and the user is aware of it, a foreground service may fit. If work can be deferred and retried, like sync or upload, WorkManager is usually better.

The weak answer is 'use WorkManager for background work' without thinking about immediacy, persistence, user visibility, and Android version restrictions."

Interview Question: Walk me through lifecycle, intents, permissions, and deep links.

Asked As / Variations

- Activity lifecycle vs Fragment lifecycle?
- Fragment view lifecycle vs Fragment lifecycle?
- Explicit vs implicit intent?
- What is a `PendingIntent`?
- How do runtime permissions affect architecture?
- What can go wrong with deep links?

Strong Answer

"I think about Android fundamentals through ownership and entry points. An Activity owns a window and goes through creation, visible, foreground, paused, stopped, and destroyed states. A Fragment has its own lifecycle, but its view has a separate lifecycle. That distinction matters because the Fragment object can outlive its view. If I keep a binding reference after `onDestroyView`, I can leak the old view or update UI that no longer exists.

Intents are messages used to request an action. An explicit intent targets a known component; an implicit intent describes an action and lets Android resolve a matching component. A `PendingIntent` is a token I give another process or the system so it can perform an action as my app later, so I treat mutability, request codes, flags, and target component carefully.

Permissions are not only UI prompts. They affect feature design. I need to handle granted, denied, permanently denied, approximate vs precise location where relevant, background restrictions, and the state where the user revokes

permission from settings.

Deep links are external entry points into the app. I validate inputs, handle missing or stale IDs, avoid trusting link parameters as authorization, and think about task/back-stack behavior. If a deep link can open an exported Activity, that Activity must be safe when launched by another app."

Tricky Follow-Up Questions And Answers

Follow-up: Why does Fragment binding get cleared?

Answer: "The Fragment may remain alive after its view is destroyed. A view binding points to the old view tree, so holding it after `onDestroyView` can leak that tree and cause stale UI access. I clear it with the view lifecycle, not the Fragment lifecycle."

Follow-up: What belongs in saved instance state?

Answer: "Small values needed to reconstruct UI: IDs, selected tab, draft text, scroll position, and simple filters. Large lists, bitmaps, repositories, and cached network responses belong in durable storage or should be reloadable."

Follow-up: What is the deep-link security answer?

Answer: "A deep link is an input boundary. I validate parameters, check authentication and authorization server-side, avoid exposing privileged screens through exported components, and make the destination robust when opened from a cold start."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: Walk me through Activity lifecycle.

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 2: Walk me through Fragment lifecycle.

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 3: What survives configuration change?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 4: What survives process death?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 5: Does ViewModel survive process death?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 6: What belongs in `SavedStateHandle`?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 7: What should go into Room or DataStore instead of saved state?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 8: Why should ViewModel not hold an Activity or View reference?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 9: Application context vs Activity context?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 10: What causes memory leaks in Android?

Senior answer: "Android leaks usually happen when an object with a longer lifetime holds an object with a shorter lifetime. Common examples are a singleton holding an Activity context, a ViewModel holding a View or Fragment binding, a callback not being unregistered, a coroutine outliving the UI scope, or Fragment view binding surviving after `onDestroyView`. I would explain it as a lifetime mismatch, not just 'forgot to clear something'. The fix is to put work in the correct scope, use application context only for app-lifetime needs, clear view references at the view lifecycle boundary, unregister listeners, and verify suspicious cases with tools like LeakCanary."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 11: How do you handle runtime permissions?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 12: What is an intent?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 13: Explicit vs implicit intent?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 14: What are deep links and what can go wrong with them?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 15: BroadcastReceiver vs Service vs WorkManager?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 16: What is the difference between `onCreate`, `onStart`, and `onResume`?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 17: Fragment view lifecycle vs Fragment lifecycle?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 18: Why should Fragment binding be cleared?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 19: What is a `PendingIntent`?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 20: What is task/back-stack behavior for deep links?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 21: What can go wrong with saving too much state in a Bundle?

Senior answer: "A `Bundle` is for small, serializable state needed to restore navigation or UI after recreation, not for large object graphs or cached data. Saving too much can cause transaction-size failures, slow recreation, stale state, and duplicated sources of truth. I would save IDs, filters, selected tabs, scroll keys, and lightweight form fields. Large lists, images, entities, and derived data belong in Room, DataStore, files, memory cache, or should be reloaded by repository policy. The senior answer is ownership: saved instance state restores the screen contract; it should not become a database or a replacement for process-death-safe persistence."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 22: How do you handle background location permission?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 23: What are exported components?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

3. Coroutines And Flow

Documentation Anchors

- Kotlin coroutines guide (https://kotlinlang.org/docs/coroutines-guide.html)
- Coroutine exception handling (https://kotlinlang.org/docs/exception-handling.html)
- Kotlin Flow (https://kotlinlang.org/docs/flow.html)
- Kotlin flows on Android (https://developer.android.com/kotlin/flow)
- Lifecycle-aware Flow collection (https://developer.android.com/topic/libraries/architecture/coroutines)
- Testing Kotlin coroutines on Android (https://developer.android.com/kotlin/coroutines/test)

Theory To Know

Coroutines and Flow are central to modern Android. Interviewers rarely stop at "what is a coroutine?" They usually continue into ownership, cancellation, exception propagation, dispatcher choice, lifecycle collection, and how streams become UI state.

The main mental model is ownership plus cancellation. A coroutine is not just async syntax; it belongs to a scope, and that scope defines when the work should stop.

Important concepts:

- coroutine vs thread
- `suspend` does not mean background
- scopes and structured concurrency
- `viewModelScope`, `lifecycleScope`, application scope
- dispatchers
- `launch`, `async`, `withContext`
- cancellation and `CancellationException`
- `coroutineScope` vs `supervisorScope`
- Flow, StateFlow, SharedFlow
- cold vs hot streams
- `flowOn`, `catch`, `collectLatest`

Interview Question: What is a coroutine?

Asked As / Variations

- Are coroutines threads?
- Does `suspend` run on a background thread?
- Why use coroutines instead of callbacks?
- What scope would you use for screen work?
- Why is `GlobalScope` discouraged?

Strong Answer

"A coroutine is a lightweight unit of asynchronous work. It is not the same as a thread. Coroutines run on threads, but they can suspend without blocking the underlying thread, which lets us write asynchronous code in a sequential style.

In Android, the important part is scope ownership. `viewModelScope` is good for work owned by a screen. If the ViewModel is cleared, that work should usually cancel. But if the work must survive the screen or process, like a retryable upload or sync, I should use a different owner, usually persistent state plus WorkManager.

Also, `suspend` does not mean background. A suspend function can run on the main thread unless the implementation switches context. For blocking I/O, I need the right dispatcher or an API that already handles threading."

Tricky Follow-Up Questions And Answers

Follow-up: Coroutine vs thread?

Answer: "A coroutine is not a thread. It runs on a thread, but it can suspend without blocking that thread. Threads are OS-level execution resources; coroutines are lightweight units of work managed by the coroutine runtime."

Follow-up: Does suspend switch threads?

Answer: "No. `suspend` only means the function can suspend and resume. It does not choose a dispatcher. A suspend function can still run on main unless it switches context internally or the caller runs it on another dispatcher."

Follow-up: Why avoid GlobalScope?

Answer: "Because it detaches work from clear ownership. If the user leaves the screen or the app state changes, `GlobalScope` work may keep running without a lifecycle owner. I prefer scopes whose lifetime matches the work."

Follow-up: What happens when ViewModel is cleared?

Answer: "Work launched in `viewModelScope` is cancelled when the ViewModel is cleared. That is correct for screen-owned work because stale requests should not keep updating a screen that no longer exists. If the work must survive the screen, like an upload or sync, it needs a different owner such as persisted state plus WorkManager."

Interview Question: How do coroutine exceptions work?

Strong Answer

"Exception behavior depends on the builder and scope. With `launch`, an unhandled exception normally propagates to the parent and can cancel sibling coroutines in a regular structured scope. With `async`, the exception is captured in the `Deferred` and is observed when I call `await`.

If child tasks are part of one all-or-nothing operation, normal `coroutineScope` behavior is useful because one failure cancels the whole operation. If child tasks are independent, I can use `supervisorScope` or `SupervisorJob`, but then I need to handle each failure deliberately.

I am also careful with `CancellationException`. Cancellation is normal coroutine control flow. If I catch broad exceptions and swallow cancellation, I can break structured concurrency and keep work running when it should stop."

Tricky Follow-Up Questions And Answers

Follow-up: launch vs async?

Answer: "`launch` returns a `Job` and is for work where the result is not directly returned. `async` returns `Deferred<T>` and is for concurrent work that produces a value. If I use `async`, I should normally `await`; otherwise I may hide exceptions or use the wrong abstraction."

Follow-up: coroutineScope vs supervisorScope?

Answer: "`coroutineScope` treats child work as one unit: if one child fails, the scope fails and siblings are cancelled. `supervisorScope` lets child failures be handled independently. I choose based on whether the operation is all-or-nothing or partially useful."

Follow-up: Why does CoroutineExceptionHandler not catch everything?

Answer: "`CoroutineExceptionHandler` is for uncaught exceptions at a coroutine boundary; it is not a general try/catch for every child coroutine. Exceptions in `async` are exposed through `await`, and child failures usually propagate through the parent scope. In practice, I still handle expected failures close to the operation that can recover from them."

Follow-up: Should you catch CancellationException?

Answer: "Usually I let it propagate. If I catch broad exceptions, I either avoid catching cancellation or rethrow it. Cancellation is normal control flow, not a business error."

Interview Question: Flow vs StateFlow vs SharedFlow?

Asked As / Variations

- Cold Flow vs hot Flow?

- Should ViewModel expose Flow or StateFlow?
- SharedFlow for events?
- How do you model navigation events?
- Why is my StateFlow not emitting duplicate values?

Strong Answer

"A regular Flow is usually cold: it starts when collected, and each collector can trigger its own execution. That works well for streams of work or transformations.

StateFlow is a hot state holder. It always has a current value, so it is a good fit for ViewModel UI state. The screen can render from the latest snapshot at any time.

SharedFlow is a hot shared stream with configurable replay and buffering. It can be useful for broadcast-like emissions, but I do not use it blindly for all one-off events. Navigation and snackbars are lifecycle-sensitive. Sometimes SharedFlow is fine; sometimes state plus acknowledgement is safer. The key is understanding replay, lifecycle, duplication, and event loss."

Tricky Follow-Up Questions And Answers

Follow-up: What does flowOn affect?

Answer: "`flowOn` changes the context of upstream operators before it. That matters because putting it in the wrong place can leave expensive work on the wrong dispatcher. It does not magically move downstream collection work."

Follow-up: What does catch catch?

Answer: "`catch` catches upstream exceptions from where it is placed. It does not catch exceptions thrown inside the final collector unless that logic is moved upstream into operators like `onEach` before `catch`."

Follow-up: How do you collect Flow safely in Android?

Answer: "I collect UI flows with lifecycle awareness, so the UI is not doing work while stopped and does not leak collectors. In Compose, I prefer lifecycle-aware collection APIs when available. The ViewModel exposes state; the UI collects and renders."

Interview Question: `stateIn` vs `shareIn`?

Strong Answer

"Both convert a cold Flow into a hot shared flow inside a scope, but they serve different purposes. `stateIn` gives me a StateFlow with a current value, so it is a natural fit for UI state exposed from a ViewModel. `shareIn` gives me a SharedFlow, which is useful when I want to share emissions without necessarily representing current state.

The important part is the scope and sharing policy. If I use `viewModelScope`, the shared stream lives with the ViewModel. With `SharingStarted.WhileSubscribed`, I can avoid doing upstream work when nobody is observing, while still keeping state briefly if configured.

The trap is turning every Flow hot too early. I use `stateIn` when I truly need state with a current value, and `shareIn` when I need shared emissions. Otherwise a regular cold Flow may be simpler."

Interview Question: Channel vs SharedFlow for events?

Strong Answer

"I do not treat UI events as a one-size-fits-all problem. A Channel can model one-consumer events, while SharedFlow can broadcast to multiple collectors with configurable replay and buffering. For UI events like navigation or snackbars, lifecycle matters more than the API name.

If the event must not be lost across configuration change, maybe it should be represented in state and acknowledged after the UI handles it. If it is okay to emit only to active collectors, a SharedFlow with no replay may be fine. If only one consumer should receive it, a Channel can fit.

The senior answer is to explain delivery semantics: replay, duplication, loss, lifecycle, and ownership."

Interview Question: How do Flow operators like `callbackFlow`, `combine`, `zip`, and `flatMapLatest` work?

Asked As / Variations

- What is `callbackFlow`?
- Why is `awaitClose` important?
- `combine` vs `zip`?
- `flatMapLatest` vs `mapLatest`?
- What is `debounce` useful for?
- How do you run two requests in parallel and combine results?

Strong Answer

"I answer Flow operators by describing the stream behavior, not by naming APIs. `callbackFlow` is for adapting callback-based APIs into a Flow, especially when values can arrive over time, like location updates, sensors, Bluetooth, or SDK listeners. The critical part is cleanup: `awaitClose` unregisters the callback when collection is cancelled. Without that, I can leak the listener or keep producing values after the UI is gone.

For combining streams, `combine` emits whenever either upstream emits after both have at least one value, using the latest value from the other stream. `zip` pairs emissions one-to-one, so it waits for the next value from each side. For UI state, `combine` is usually more natural; for strict pairs, `zip` can make sense.

For latest-work patterns, `flatMapLatest` switches to a new inner Flow and cancels the previous one when the upstream changes. That is useful for search queries, selected filters, or changing IDs. `mapLatest` is similar when the transformation itself is suspendable and the previous transform should cancel. `debounce` waits for input to settle before emitting, which is common for search boxes.

For parallel requests, I usually use `coroutineScope` with `async` when I need two suspend calls concurrently and one combined result. For streams, I use Flow operators. I choose based on whether the problem is one-shot concurrency or ongoing reactive state."

Tricky Follow-Up Questions And Answers

Follow-up: What does callbackFlow need?

Answer: "It needs a clear registration and cleanup story: register the callback inside the builder, emit with `trySend` or `send`, and call `awaitClose` to unregister. If cleanup is missing, cancellation does not clean the external callback."

Follow-up: combine vs zip?

Answer: "`combine` reacts to the latest values and emits whenever either source updates after initial values exist. `zip` pairs values by position. `combine` fits UI state; `zip` fits paired workflows."

Follow-up: flatMapLatest vs mapLatest?

Answer: "`flatMapLatest` switches to a new Flow and cancels the previous Flow. `mapLatest` cancels the previous transform block. I use `flatMapLatest` when the new input changes the whole stream, like query to search results."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: What is a coroutine?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 2: Are coroutines threads?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 3: Does `suspend` mean background thread?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 4: What is structured concurrency?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 5: `launch` vs `async` vs `withContext`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 6: What does `async` return?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 7: What happens if you call `async` and never `await`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 8: What happens when a child coroutine fails?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 9: `coroutineScope` vs `supervisorScope`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 10: What is `CancellationException`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 11: Why should you not swallow cancellation?

Senior answer: "I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered:

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

Question 12: Why is `GlobalScope` discouraged?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 13: `Dispatchers.IO` vs `Dispatchers.Default`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 14: What is Flow?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 15: Cold Flow vs hot Flow?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 16: Flow vs StateFlow vs SharedFlow?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 17: How do you model one-off events?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 18: What does `flowOn` affect?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 19: What does `catch` catch?

Senior answer: "In Flow, `catch` catches exceptions from upstream of where the operator is placed. It does not catch exceptions thrown downstream by the collector, and it should not be used to hide cancellation. A senior answer names placement: `source.map { ... }.catch { ... }.collect { ... }` catches failures from `source` and `map`, but not failures inside `collect`. If I recover, I emit a fallback state or map the failure into a typed error. If the failure is `CancellationException`, I normally let it propagate because cancellation is part of structured concurrency, not an ordinary business error."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 20: `collect` vs `collectLatest`?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 21: `stateIn` vs `shareIn`?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 22: How do you collect Flow safely in Android?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 23: What dispatcher should CPU-heavy work use?

Senior answer: "I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered:

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

Question 24: What dispatcher should blocking I/O use?

Senior answer: "I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered:

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

Question 25: What happens if you block `Dispatchers.Main`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 26: How do you run two requests in parallel and combine results?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 27: What is `NonCancellable` used for?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 28: What is `callbackFlow`?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 29: Why is `awaitClose` important?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 30: What is `debounce` useful for?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 31: `combine` vs `zip`?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 32: `flatMapLatest` vs `mapLatest`?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

4. Jetpack Compose

Documentation Anchors

- Thinking in Compose (https://developer.android.com/develop/ui/compose/mental-model)
- State and Jetpack Compose (https://developer.android.com/develop/ui/compose/state)
- Compose side effects (https://developer.android.com/develop/ui/compose/side-effects)
- Lifecycle-aware state collection in Compose (https://developer.android.com/develop/ui/compose/state#use-other-types-of-state-in-jetpack-compose)
- Compose performance (https://developer.android.com/develop/ui/compose/performance)
- Testing Compose (https://developer.android.com/develop/ui/compose/testing)

Theory To Know

Compose interviews test whether you understand declarative UI, state, recomposition, side effects, lifecycle, and performance. Knowing composable syntax is not enough. You need to explain how state drives UI and why side effects must be controlled.

The mental model is: UI is a function of state. When state changes, Compose may re-run composables that read that state. Your code must remain correct even if recomposition happens often.

Important concepts:

- UI as a function of state
- recomposition
- state hoisting
- `remember`, `rememberSaveable`, ViewModel state
- side effects: `LaunchedEffect`, `DisposableEffect`, `SideEffect`
- `rememberUpdatedState`
- mutable state pitfalls
- lazy list keys
- stability and skipping

Interview Question: What is recomposition?

Asked As / Variations

- What triggers recomposition?
- Does Compose redraw the whole screen?
- Why should composables be side-effect free?
- Why did my UI not update after changing a list?
- How do you reduce unnecessary recomposition?

Strong Answer

"Recomposition is Compose re-invoking composable functions whose observed state may have changed, so it can produce an updated UI description. It is not the same as redrawing the whole screen. Compose can recompose only affected parts and can skip work when inputs are stable and unchanged.

Because composables may be called many times, they should be side-effect free. I should not put network calls, database writes, or navigation directly in the composable body. Those belong in ViewModel logic or controlled effect APIs like `LaunchedEffect`.

If a UI does not update, I check whether the value is observable Compose state. Mutating a regular mutable list in place may not trigger recomposition. I usually prefer immutable state updates or snapshot-aware state containers."

Tricky Follow-Up Questions And Answers

Follow-up: Does recomposition redraw everything?

Answer: "No. Recomposition re-invokes composable functions that may need to produce new UI. Compose can skip work when inputs are stable and unchanged. Redrawing/rendering is a later step; recomposition is about recalculating UI description."

Follow-up: What triggers recomposition?

Answer: "A composable can recompose when state it read changes or when its parent passes changed parameters. The practical detail is that Compose tracks reads of observable state, not arbitrary mutation. If I mutate a normal list in place or change data outside Compose-aware state, Compose may not know it should update."

Follow-up: Why did mutating a list not update UI?

Answer: "If the list is a regular mutable list and I mutate it in place, Compose may not observe a new state value. I usually replace the list with a new immutable value or use snapshot-aware state containers."

Follow-up: When does LaunchedEffect restart?

Answer: "It restarts when one of its keys changes. The key should represent the lifetime of the effect. A wrong key can cause repeated work or stale captured values."

Interview Question: What is state hoisting?

Strong Answer

"State hoisting means moving state to the owner that needs to read or modify it. It does not mean putting all state in the ViewModel. Some state is local UI state; some belongs to the screen state holder.

The usual pattern is state down, events up. A composable receives values and callbacks instead of owning business logic directly. That makes it easier to preview, test, and reuse.

The decision depends on lifetime. Temporary visual state can stay local. Screen state, validation, loading, and errors usually belong in ViewModel. Durable data belongs below that, in the data layer."

Interview Question: What is `LaunchedEffect`, and when does it restart?

Asked As / Variations

- Why is `LaunchedEffect` running multiple times?
- What do effect keys mean?
- `LaunchedEffect(Unit)` good or bad?
- How do you avoid stale lambdas in effects?

Strong Answer

"`LaunchedEffect` starts a coroutine tied to the composable's lifecycle in the composition. It runs when the composable enters composition, and it cancels and restarts when one of its keys changes.

The key is the important part. If I key it with `Unit`, it runs for that composition lifetime. If I key it with `userId`, it restarts when `userId` changes. A wrong key can cause repeated network calls, repeated navigation, or stale values captured inside the coroutine.

I use `LaunchedEffect` for UI side effects that are truly tied to composition, like collecting one-off effects, triggering an animation, or reacting to a parameter change. I do not use it as a place to hide business logic that belongs in the ViewModel."

Tricky Follow-Up Questions And Answers

Follow-up: When use rememberUpdatedState?

Answer: "When a long-running effect needs the latest lambda or value without restarting the effect. It helps avoid stale captures while keeping the effect lifetime stable."

Follow-up: DisposableEffect?

Answer: "I use `DisposableEffect` when I need setup and cleanup tied to composition, like registering and unregistering a listener."

Interview Question: `remember` vs `rememberSaveable` vs ViewModel?

Strong Answer

"I choose based on lifetime. `remember` survives recomposition while the composable remains in composition. `rememberSaveable` can survive configuration change and process recreation for values that can be saved. ViewModel survives configuration change and owns screen state and business interactions, but the object itself does not survive process death.

For small UI state like a selected tab or text field, `rememberSaveable` can be enough. For screen state that involves loading, validation, repositories, or business behavior, I prefer ViewModel. For durable data, I go below UI entirely: Room, DataStore, files, or backend.

The trap is using one storage mechanism for everything. Compose state, ViewModel state, saved state, and persistent storage solve different lifetime problems."

Interview Question: What are stability, skippability, `derivedStateOf`, and LazyColumn keys?

Asked As / Variations

- What makes a type stable in Compose?
- What is skippability?
- When should you use `derivedStateOf`?
- When is `derivedStateOf` overused?
- How do LazyColumn keys help?
- How do you test Compose semantics?

Strong Answer

"Compose performance is not only 'avoid recomposition.' Recomposition is normal. The goal is to make recomposition correct, scoped, and not unnecessarily expensive.

Stability is Compose's ability to reason that a value's public behavior will not change unexpectedly without Compose being notified. If parameters are stable and unchanged, a composable may be skippable, meaning Compose can skip re-running it. This is why immutable UI state and clear state ownership matter. Passing unstable objects, mutating lists in place, or capturing large objects in lambdas can make skipping harder or behavior less predictable.

`derivedStateOf` is for cases where inputs change more often than the UI actually needs to update. A common example is deriving a boolean from scroll position. It is not a generic 'computed property' tool; overusing it adds complexity and can make state harder to understand.

Lazy list keys give item identity. Without stable keys, remembered item state can move incorrectly when items are inserted, removed, or reordered. With keys, Compose can keep item state associated with the item, not just the index.

For testing, I prefer testing behavior through semantics: visible text, content descriptions, state descriptions, clicks, and navigation effects. I do not test whether a composable recomposed a certain number of times unless I am specifically diagnosing performance."

Tricky Follow-Up Questions And Answers

Follow-up: Why can List<T> be tricky?

Answer: "A regular `List<T>` interface does not guarantee immutability to Compose. Also, mutating the same list instance in place may not notify state observers. I prefer replacing state with a new immutable value or using snapshot-aware containers deliberately."

Follow-up: When is derivedStateOf overused?

Answer: "When the derived value changes at the same rate as the inputs or when a simple expression is enough. It helps when it prevents unnecessary recompositions caused by high-frequency input changes."

Follow-up: How do LazyColumn keys help?

Answer: "Keys preserve item identity across dataset changes. If an item has remembered state and the list changes before it, the key lets Compose move that state with the item instead of associating it with the old index."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: What is recomposition?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 2: Does recomposition redraw the whole screen?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 3: What triggers recomposition?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 4: What is state hoisting?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 5: Does all state belong in ViewModel?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 6: `remember` vs `rememberSaveable`?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 7: What should survive process death in Compose?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 8: What is `LaunchedEffect`?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 9: When does `LaunchedEffect` restart?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 10: What is `DisposableEffect`?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 11: What is `rememberUpdatedState` for?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 12: Why can mutating a list not update Compose UI?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 13: What are lazy list keys?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 14: How do you handle navigation events in Compose?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 15: How do you investigate Compose performance?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 16: What makes a type stable in Compose?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 17: What is skippability?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 18: What is `derivedStateOf` for?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 19: When can `derivedStateOf` be overused?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 20: How do LazyColumn keys help?

Senior answer: "I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered:

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

Question 21: How do you avoid repeated navigation in Compose?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Question 22: How do you test Compose semantics?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 23: What is snapshot state?

Senior answer: "I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered:

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

5. Architecture

Documentation Anchors

- Android app architecture guide (https://developer.android.com/topic/architecture)
- Android architecture recommendations (https://developer.android.com/topic/architecture/recommendations)
- UI layer guide (https://developer.android.com/topic/architecture/ui-layer)
- Data layer guide (https://developer.android.com/topic/architecture/data-layer)
- Domain layer guide (https://developer.android.com/topic/architecture/domain-layer)
- Offline-first data layer (https://developer.android.com/topic/architecture/data-layer/offline-first)

Theory To Know

Architecture questions test ownership, dependency direction, testability, modularity, and trade-offs. A senior answer should not sound like "I use MVVM, Repository, Hilt." It should explain why each layer exists and what problem it solves.

The mental model is responsibility. UI renders, ViewModel owns screen state, repositories own data policy, use cases coordinate business operations when they add value, and DI owns construction/lifetimes.

Important concepts:

- MVVM
- MVI
- Clean Architecture
- Repository
- UseCase
- unidirectional data flow
- single source of truth
- dependency injection
- modularization
- legacy migration

Interview Question: Explain MVVM in Android.

Asked As / Variations

- What architecture do you use in Android?
- What belongs in ViewModel?
- Where does business logic live?
- How do you avoid ViewModel bloat?
- MVVM vs MVI?

Strong Answer

"In Android, I treat MVVM as a presentation architecture. The UI renders state and sends user actions. The ViewModel owns screen state, handles screen events, and coordinates with use cases or repositories. The data layer handles where data comes from and how it is cached or synchronized.

The benefit is separation of responsibilities. The UI does not need to know how data is loaded, and the ViewModel does not need to know how the UI is drawn. That makes the screen easier to test because I can drive the ViewModel with events and assert state.

The trap is putting everything in the ViewModel. If it handles API mapping, database policy, business validation, analytics, and navigation all together, it becomes a god object. For simple screens, MVVM can stay lightweight. For complex flows, I may introduce use cases, reducers, mappers, or coordinators based on the real complexity."

Interview Question: What is Clean Architecture?

Strong Answer

"Clean Architecture is about dependency direction and separation of responsibilities. The domain or business rules should not depend on Android UI classes, Retrofit DTOs, Room entities, or Compose. Outer layers can know about frameworks, but core rules should remain testable and independent.

In Android, this often becomes UI, ViewModel, domain/use cases, repositories, and data sources. But the folders are not the architecture. The important question is whether the boundaries reduce coupling and make change safer.

It becomes overkill when the layers are only pass-through wrappers. A use case that only calls one repository method may not add value unless the team wants strict consistency. I add structure when it solves real problems: complex orchestration, shared business rules, multiple data sources, testability, or team/module boundaries."

Tricky Follow-Up Questions And Answers

Follow-up: When do you use UseCases?

Answer: "I use a use case when it represents a meaningful operation: combining repositories, enforcing business rules, orchestrating several steps, or making logic reusable and testable. I avoid creating use cases that only pass one repository call through unless the project convention is worth the ceremony."

Follow-up: What is single source of truth?

Answer: "It means one authoritative place represents the current state of some data. In offline-first Android, Room is often the source of truth for cached entities, and network refresh writes into Room instead of the UI juggling network and database state separately."

Follow-up: How do you avoid ViewModel bloat?

Answer: "I split responsibilities when the ViewModel starts doing unrelated work: business rules, mapping, sync policy, analytics, and UI state all together. Sometimes the right extraction is a use case; sometimes it is a mapper, reducer, or coordinator."

Interview Question: How do dependency injection, Hilt scopes, and modularization fit architecture?

Asked As / Variations

- What is dependency inversion?
- Hilt vs Dagger?
- Hilt vs Koin?
- What are Hilt scopes?
- What should be singleton scoped?
- Why qualify dispatchers in DI?
- How would you modularize a large Android app?

Strong Answer

"Dependency inversion means higher-level policy should not depend directly on lower-level implementation details. In Android, that usually means ViewModels depend on interfaces or stable abstractions, while data modules provide Retrofit, Room, DataStore, and concrete repositories.

DI is the construction mechanism. Hilt builds on Dagger and gives Android-aware integration for common components like Activity, Fragment, ViewModel, and Worker injection. Dagger is more manual and flexible. Koin is runtime and simpler to set up, but it does not give the same compile-time graph guarantees as Dagger/Hilt.

Scopes are lifetime boundaries. A singleton-scoped dependency should be safe for the whole app process, such as OkHttp, Retrofit, Room, or a repository that is designed as process-wide. I avoid singleton-scoping anything that captures Activity context, screen state, or user-session assumptions that must be reset on logout.

I like qualifying dispatchers in DI because it makes threading policy explicit and testable: `IoDispatcher`, `DefaultDispatcher`, `MainDispatcher`. Hardcoded dispatchers make coroutine tests and performance fixes harder.

For modularization, I start from product and dependency boundaries, not folders. Feature modules can own screens and feature-specific logic. Core modules can own networking, database, design system, analytics, and common testing tools. The goal is faster builds, clearer ownership, and safer boundaries; if modules only add Gradle complexity without ownership benefits, they are not helping."

Tricky Follow-Up Questions And Answers

Follow-up: What should be singleton scoped?

Answer: "Only dependencies that are safe and intended to live for the app process: database, HTTP client, API services, DataStore, and stateless or carefully stateful repositories. I do not singleton-scope Activity references, bindings, adapters, or screen-specific state."

Follow-up: How do you replace dependencies in tests?

Answer: "I design dependencies so tests can replace network, database, dispatchers, clocks, and repositories with fakes. With DI, that usually means test modules or constructor injection. The test should control time, inputs, and failure modes."

Follow-up: How do you inject workers?

Answer: "With Hilt, I use the WorkManager/Hilt integration so Workers can receive dependencies through assisted injection. I still persist the work input because a Worker can run after process recreation."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: Explain MVVM in Android.

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 2: MVVM vs MVI?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 3: What is Clean Architecture?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 4: When is Clean Architecture overkill?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 5: What is the Repository pattern?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 6: What should not go into a Repository?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 7: When do you use UseCases?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 8: What is unidirectional data flow?

Senior answer: "I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered:

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Question 9: What is single source of truth?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 10: DTO vs domain model vs UI model?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 11: How do you avoid ViewModel becoming too large?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 12: How would you migrate a legacy MVP/XML app?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 13: How would you modularize a large Android app?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 14: Hilt vs Dagger?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 15: Hilt vs Koin?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 16: What are Hilt scopes?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 17: What should be singleton scoped?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 18: Why qualify dispatchers in DI?

Senior answer: "I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered:

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

Question 19: How do you inject workers?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 20: How do you replace dependencies in tests?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 21: What is dependency inversion?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

6. Design Patterns

Documentation Anchors

- Dependency injection in Android (https://developer.android.com/training/dependency-injection)
- Hilt dependency injection (https://developer.android.com/training/dependency-injection/hilt-android)
- Guide to app architecture (https://developer.android.com/topic/architecture)

Theory To Know

Design patterns are useful when they solve a recurring design problem. They are not something to force into every class.

Important Android patterns:

- Repository
- Observer
- Strategy
- Factory
- Adapter
- Singleton
- State
- Command
- Decorator
- Dependency Injection

Interview Question: What design patterns have you used in Android?

Strong Answer

"I would answer by problem, not by listing pattern names. For data boundaries, I use Repository so UI and domain code do not care whether data comes from network, database, cache, or sync. For reacting to state changes, Android uses observer-like streams through Flow, StateFlow, LiveData, or Compose state.

For object creation, I usually prefer dependency injection, but factories still make sense when creation depends on runtime values. Strategy is useful when behavior varies, like validation or sync conflict policy. Adapter appears both in

framework APIs, like RecyclerView Adapter, and at boundaries, like wrapping callback APIs into Flow or mapping DTOs into domain models.

The senior point is not to force patterns. Patterns help when they reduce coupling, isolate change, or make behavior explicit. They hurt when they create unnecessary layers."

Interview Question: Singleton vs dependency injection?

Strong Answer

"Singleton describes lifetime: one shared instance. Dependency injection describes how objects receive dependencies. Those are different ideas. I can have a DI-managed singleton, like a single Room database or OkHttp client, without exposing it as a global static object.

The problem with global singletons is hidden ownership. Classes can reach into global state without declaring what they need, tests become harder, and it is easy to leak Activity context if the singleton holds UI references.

With Hilt or Dagger, the graph controls the lifetime and makes dependencies explicit. So I am not against singleton lifetime; I am against uncontrolled global access."

Interview Question: What is pattern abuse?

Strong Answer

"Pattern abuse is when the pattern becomes more important than the problem. For example, adding a UseCase, Factory, Manager, Mapper, and Interface around a one-line operation may make the code look architected but harder to understand.

In Kotlin, some classic Java patterns are less necessary because functions, lambdas, sealed classes, data classes, and default parameters can solve the same problem more simply.

My rule is that a pattern should reduce coupling, isolate change, or clarify behavior. If it only adds ceremony, I avoid it."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: What design patterns have you used in Android?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 2: Singleton: when is it okay and when is it dangerous?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 3: Factory vs dependency injection?

Senior answer: "I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered:

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Question 4: Adapter pattern examples in Android?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 5: Observer pattern in Android?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 6: Strategy pattern example in Android?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 7: State pattern example in Android?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 8: Command pattern for offline operations?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 9: How do Kotlin features reduce the need for some classic patterns?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

Question 10: What is pattern abuse?

Senior answer: "I would avoid reciting a pattern catalog and instead name the problem the pattern solves. Observer appears in Flow, LiveData, and UI state observation. Adapter maps one interface or model shape to another. Strategy swaps behavior such as sorting, validation, or retry policy. Factory centralizes object creation when construction has variants. Command can represent persisted offline operations. Singleton is acceptable for true process-wide stateless or coordinated resources, but dependency injection is better for testability and lifetime control. The senior answer also warns against pattern abuse: if the pattern hides simple code or ownership, it is hurting the design."

Tricky follow-ups answered:

Follow-up: How should you present a pattern?

Answer: Name the problem first, then the pattern. Do not recite pattern names without a reason.

Follow-up: When is Singleton okay?

Answer: For true process-wide stateless or coordinated resources. Avoid it for hidden mutable state and hard-to-replace dependencies.

Follow-up: How does Kotlin change patterns?

Answer: Sealed classes, higher-order functions, extension functions, and data classes can replace some verbose classic pattern implementations.

Follow-up: What is pattern abuse?

Answer: Adding indirection that hides simple behavior, ownership, or data flow without reducing real complexity.

7. Mobile System Design

Documentation Anchors

- Offline-first data layer (https://developer.android.com/topic/architecture/data-layer/offline-first)
- WorkManager overview (https://developer.android.com/topic/libraries/architecture/workmanager)
- Background work overview (https://developer.android.com/develop/background-work/background-tasks)
- Save data in a local database using Room (https://developer.android.com/training/data-storage/room)
- Paging library overview (https://developer.android.com/topic/libraries/architecture/paging/v3-overview)

Theory To Know

Mobile system design is not backend-only design. You must account for unreliable network, process death, battery, storage, background limits, UI state, and release risk.

Important concepts:

- local source of truth
- offline-first
- cached reads vs offline writes
- sync and conflict resolution
- WorkManager vs foreground service
- idempotency
- pagination

- token refresh
- modularization

Interview Question: Design an offline-first feed.

Asked As / Variations

- Design a feed that works offline.
- How do you cache paginated data?
- What is the source of truth?
- How do you handle likes while offline?
- What happens if sync fails?

Strong Answer

"I would first clarify what offline-first means for this product. Cached reading is different from offline writing. For a feed, I would usually make local storage the source of truth. The UI observes paged data from Room, and network refresh writes into Room inside a transaction. That keeps rendering consistent because the UI has one source of truth.

For pagination, I need stable remote keys or cursors so refresh and append do not create duplicates. If the network fails and cached data exists, I would show cached content with a non-blocking error or stale indicator.

If users can perform actions offline, such as like or save, I need more than cache. I need a pending operation queue, optimistic UI rules, retry policy, idempotent request IDs, and conflict handling. WorkManager can drain the queue when constraints are met. I would also test process death during sync."

Interview Question: Design photo upload with retry.

Strong Answer

"I would persist upload state before starting the upload. Each selected file becomes an upload record with a local ID, file reference, status, progress, retry count, and maybe a server correlation ID. The UI observes those records, so progress survives Activity recreation.

For execution, if the upload can be deferred and must survive process death, WorkManager is a strong default because it supports constraints and retry. If the upload is immediate and user-visible for a long time, I would evaluate foreground service behavior and OS restrictions.

The hardest part is avoiding duplicates. Retries need idempotent upload IDs or server support so a repeated request does not create multiple files. I also need to handle logout, deleted local files, partial failures, network changes, and cancellation. I would test poor network, process death during upload, retry after restart, and duplicate prevention."

Interview Question: Design token refresh architecture.

Strong Answer

"I would keep auth out of ViewModels and centralize it in the data/network layer. Usually an OkHttp interceptor attaches the access token, and an authenticator or coordinated auth component handles 401 responses and refresh.

The important part is concurrency. If ten requests fail with 401 at the same time, I do not want ten refresh calls. I would serialize refresh or share the refresh result so pending requests wait for one refresh operation. I also avoid refreshing the refresh endpoint itself and define what happens if refresh fails: clear auth state and route the user to login.

I treat tokens as sensitive but also remember the client is not a trusted environment. Server-side authorization still matters."

Interview Question: Design chat, startup, pagination, or feature flags for Android.

Asked As / Variations

- Design a chat feature with retry.
- Design pagination with cache invalidation.
- Design app startup for a large app.
- Design feature flags for mobile.
- How do you handle logout with pending offline work?
- How do you monitor sync failures?

Strong Answer

"For mobile system design, I first separate product guarantees. A chat feature is not just a list of messages; it needs local IDs, optimistic sends, delivery status, retry, deduplication, ordering, pagination, push notification behavior, and reconciliation with server truth. The UI should observe local state, while sync writes server results back into the local source of truth.

For pagination, I care about stable keys, refresh behavior, invalidation, duplicate prevention, and what the UI shows when append fails but cached items exist. A good answer names both data consistency and user experience.

For startup, I split critical path from deferrable work. Authentication state, feature gates required for routing, and essential local data may be needed early. Analytics setup, non-critical SDKs, warmups, and sync can often be deferred or moved behind App Startup/WorkManager depending on the requirement. I measure cold start instead of guessing.

For feature flags, I think about defaults, caching, targeting, offline behavior, kill switches, and version compatibility. The app must behave safely if flags are stale or unavailable.

Logout is a common senior follow-up. I clear sensitive local state, cancel or mark user-owned pending work, remove tokens, reset in-memory session state, and make sure background workers cannot upload data for the wrong user after logout."

Tricky Follow-Up Questions And Answers

Follow-up: How do you avoid duplicate writes?

Answer: "Use idempotency keys, local operation IDs, server correlation IDs, and transactional local state changes. Retries should be safe to repeat."

Follow-up: How do you monitor sync failures?

Answer: "I track retry counts, terminal failures, queue age, error categories, and user-visible recovery paths. A sync system that fails silently is not production-ready."

Follow-up: What makes mobile system design different from backend system design?

Answer: "Mobile has process death, lifecycle, storage limits, battery, background execution restrictions, unreliable network, app upgrades, and UI state restoration. Backend scale still matters, but Android interviews expect device constraints too."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: Design an offline-first feed.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 2: Design an offline notes app.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 3: Design a chat feature.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 4: Design photo upload with retry.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 5: Design background sync.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 6: WorkManager vs foreground service?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 7: How do you handle offline writes?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 8: How do you handle sync conflicts?

Senior answer: "I would first ask what a conflict means for the product, because the right policy is domain-specific. Some data can use last-write-wins, some should be server-authoritative, some can merge field by field, and some must ask the user. Technically, I persist enough metadata to detect conflict: operation IDs, local version, remote version, updated timestamps when they are meaningful, and server response state. The UI should expose conflicted or failed states instead of silently dropping work. A senior answer also mentions idempotency, retries, monitoring conflict rate, and a recovery path after process death or logout."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 9: How do you avoid duplicate writes?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 10: How do you handle logout with pending offline work?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 11: Design token refresh architecture.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 12: Design app startup for a large app.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 13: Design feature flags for mobile.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 14: Design offline-first notes with conflict resolution.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 15: Design chat message sending with retry.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 16: Design pagination with cache invalidation.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 17: Design feature flags for Android.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 18: Design app startup initialization.

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 19: How do you handle user logout in an offline-first app?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 20: How do you clean pending work after logout?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Question 21: How do you monitor sync failures?

Senior answer: "I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered:

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

8. Testing

Documentation Anchors

- Testing Kotlin coroutines on Android (https://developer.android.com/kotlin/coroutines/test)
- Test apps on Android (https://developer.android.com/training/testing)
- Compose testing (https://developer.android.com/develop/ui/compose/testing)
- Room migration testing (https://developer.android.com/training/data-storage/room/migrating-db-versions)

Theory To Know

Senior testing questions focus on strategy, asynchronous code, Flow emissions, UI behavior, fakes, and migration safety.

Important concepts:

- unit vs integration vs UI tests
- `runTest`
- dispatcher injection
- ViewModel state tests
- Flow testing
- fakes vs mocks
- Compose semantics
- Room migration tests

- CI flakiness

Interview Question: How do you test a ViewModel that uses coroutines and Flow?

Asked As / Variations

- How do you test loading then success?
- How do you avoid `Thread.sleep` in coroutine tests?
- How do you test StateFlow?
- How do you test errors and retry?
- Fakes or mocks?

Strong Answer

"I test the ViewModel through public behavior, not private methods. I create it with fake dependencies, trigger public events, and assert emitted UI state. For coroutine code, I use `runTest` so the test controls virtual time instead of sleeping. I also inject dispatchers or a dispatcher provider, because hardcoded dispatchers make tests harder to control.

For state, I assert the sequence: initial state, loading, success or error. If the ViewModel exposes StateFlow, I remember it has an initial value. If the Flow never completes, I collect it in a test coroutine and cancel it deliberately.

The main thing I want from these tests is confidence in behavior: retry, validation, network failure, cached data, and cancellation. I do not want tests that only verify internal method calls unless that interaction is the behavior."

Interview Question: How do you test Flow emissions?

Strong Answer

"I first ask what kind of Flow I am testing. If it is a simple cold Flow with one value, `first()` may be enough. But for UI state, I usually care about the sequence: initial, loading, content, error, retry, and so on.

For StateFlow, I remember there is always an initial value. For flows that never complete, I collect in a test coroutine and cancel the collection. I use fake dependencies that can emit values, delays, and errors deliberately, instead of relying on real timing.

The weak test only checks the final state. A better test catches transition bugs: loading never appears, cached content disappears on error, retry emits duplicate states, or cancellation leaves collectors alive."

Interview Question: How do you test Room migrations?

Strong Answer

"I test migrations with old schema data, not just by opening the database. The test should create the database at an older version, insert representative data, run the migration, validate the new schema, and verify the data still means the same thing.

I avoid destructive migration unless the data is truly disposable. For user-created data, pending offline operations, or important cached state, destructive migration is a product decision, not a convenience.

The reason this matters is that local data migrations are hard to undo. If a release corrupts user data, a rollback may not fully restore the old local database."

Interview Question: What are `MainDispatcherRule`, test dispatchers, Turbine, and WorkManager testing?

Asked As / Variations

- What is `MainDispatcherRule`?
- `StandardTestDispatcher` vs `UnconfinedTestDispatcher`?
- What does `advanceUntilIdle()` do?
- How do you test retry with virtual time?
- What is Turbine used for?
- How do you test `stateIn`?
- How do you test WorkManager?

Strong Answer

"Coroutine tests should control time and dispatching. `MainDispatcherRule` is a JUnit rule pattern that replaces `Dispatchers.Main` during tests and resets it afterward. That matters for ViewModels because `viewModelScope` uses Main by default.

`StandardTestDispatcher` is predictable because work runs when the scheduler advances; it is good for tests where I want explicit control. `UnconfinedTestDispatcher` starts work more eagerly and can be convenient, but it can hide ordering issues if overused. `advanceUntilIdle()` runs queued work until there is nothing immediately left, while virtual-time APIs let me test delays, debounce, and retry without sleeping.

Turbine is a common Flow testing library. It lets me collect emissions and assert them sequentially, including initial StateFlow values, later emissions, completion, or errors. For `stateIn`, I remember the sharing policy matters: if the upstream starts only while subscribed, the test must actually collect.

For WorkManager, I use WorkManager's testing utilities or integration-style tests with controlled constraints and fake dependencies. I assert the persisted work behavior, not only that a method was called. WorkManager tests should cover retry, input data, failure, cancellation, and what happens after process recreation as much as practical."

Tricky Follow-Up Questions And Answers

Follow-up: How do you avoid real delays?

Answer: "Use `runTest` and test dispatchers with virtual time. I do not use `Thread.sleep` for coroutine behavior because it makes tests slow and flaky."

Follow-up: How do you test StateFlow initial state?

Answer: "StateFlow always has a current value. I assert the initial value deliberately, then trigger events and assert the next states. If using Turbine, I expect the initial item first."

Follow-up: What makes UI tests flaky?

Answer: "Real network, real clocks, animations, uncontrolled dispatchers, shared state between tests, waiting by sleep, and assertions that depend on implementation timing instead of visible behavior."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: How do you test a ViewModel with coroutines?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 2: How do you test Flow emissions?

Senior answer: "I test Flow by collecting it in a controlled coroutine test and asserting emissions in order. Turbine is useful because it lets me `awaitItem`, assert completion or errors, and check that no unexpected events arrived. For `StateFlow`, I remember there is always an initial value, so the test should account for that before later emissions. For never-ending flows, I cancel the collection or use Turbine's cancellation helpers so the test does not hang. The senior detail is operator and lifecycle awareness: I control upstream fakes, virtual time for debounce/retry, and I assert behavior rather than the internal chain of operators."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 3: How do you test StateFlow initial state?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 4: How do you avoid real delays in tests?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 5: What is dispatcher injection?

Senior answer: "I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered:

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

Question 6: Fakes vs mocks?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 7: How do you test Compose UI?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 8: How do you test navigation behavior?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 9: How do you test Room migrations?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 10: How do you reduce flaky UI tests in CI?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 11: What is `MainDispatcherRule`?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 12: `StandardTestDispatcher` vs `UnconfinedTestDispatcher`?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 13: What does `advanceUntilIdle()` do?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 14: How do you test retry with virtual time?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 15: What is Turbine used for?

Senior answer: "I test Flow by collecting it in a controlled coroutine test and asserting emissions in order. Turbine is useful because it lets me `awaitItem`, assert completion or errors, and check that no unexpected events arrived. For `StateFlow`, I remember there is always an initial value, so the test should account for that before later emissions. For never-ending flows, I cancel the collection or use Turbine's cancellation helpers so the test does not hang. The senior detail is operator and lifecycle awareness: I control upstream fakes, virtual time for debounce/retry, and I assert behavior rather than the internal chain of operators."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 16: How do you test `stateIn`?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 17: How do you test WorkManager?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Question 18: What makes UI tests flaky?

Senior answer: "I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered:

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

9. Performance, Security, And Release

Documentation Anchors

- Android performance (https://developer.android.com/topic/performance)
- Measure app performance (https://developer.android.com/topic/performance/measuring-performance)
- Android memory guide (https://developer.android.com/topic/performance/memory)
- Security best practices (https://developer.android.com/privacy-and-security/security-best-practices)
- Android cryptography (https://developer.android.com/privacy-and-security/cryptography)
- Shrink, obfuscate, and optimize your app (https://developer.android.com/build/shrink-code)

Theory To Know

Senior Android developers are expected to diagnose production issues and understand release risk.

Important concepts:

- ANRs and main thread blocking
- jank and frame rendering
- memory leaks
- startup performance
- Android Vitals
- token storage
- client trust boundaries
- certificate pinning trade-offs
- R8, keep rules, mapping files
- staged rollout and hotfix

Interview Question: How do you investigate app jank or freezes?

Asked As / Variations

- Users say the app freezes. What do you check?
- How do you investigate ANRs?
- How do you diagnose slow startup?
- How do you debug memory leaks?
- What tools do you use for performance?

Strong Answer

"I start by measuring instead of guessing. If users report jank or freezes, I want to know whether the main thread is blocked, frames are missed during rendering, disk or database work is happening on the main thread, image loading is too heavy, or allocation pressure is causing GC.

Locally I would use Android Studio profiler, traces, and depending on the case Perfetto or Macrobenchmark. In production I would check Android Vitals, ANRs, crash reports, device distribution, and any performance metrics we have.

I avoid vague fixes like 'move it to background' without knowing the bottleneck. If the issue is recomposition, the fix differs from a slow SQL query, a large bitmap, startup initialization, or lock contention."

Interview Question: What can go wrong in release builds?

Strong Answer

"Release builds can behave differently from debug builds. R8 can shrink, optimize, and obfuscate code. That is useful, but it can break reflection, serialization, dependency injection, or framework entry points if keep rules are wrong.

I want critical flows tested in a release-like build, not only debug. That includes minification, resource shrinking, signing config, build variants, feature flags, and backend environments. I also want mapping files preserved so obfuscated crash reports can be understood.

For rollout, I prefer staged release when possible. If there is a crash spike or ANR regression, the team should stop rollout, identify the release diff, hotfix or rollback if available, and add a prevention test or checklist item."

Interview Question: How do you store tokens securely?

Strong Answer

"I start from the assumption that the mobile client is not a fully trusted environment. Anything shipped in the APK can eventually be inspected, so API keys in the client are not true secrets. Real authorization must be enforced server-side.

For tokens, I keep them out of ViewModel and UI state. They belong in the auth/data layer. Storage depends on sensitivity and product requirements, but I would use platform-backed secure storage where appropriate, avoid logging tokens or PII, and keep token lifetime and scope limited when possible.

I also think about operational behavior: refresh token failure, logout, device compromise, backups, and whether tokens should be invalidated server-side. Secure storage is one part of the system, not the entire security model."

Interview Question: Certificate pinning: good idea or risk?

Strong Answer

"Certificate pinning can reduce some man-in-the-middle risk, but it is not something I add casually. The operational risk is real: certificates rotate, intermediates change, and a bad pinning setup can lock users out of the API.

I would use pinning only when the threat model justifies it, and I would plan backup pins, rotation, monitoring, and an emergency strategy. I would also remember that pinning does not make the client trusted; server-side authorization and secure API design still matter.

So my answer is not yes or no. It is a trade-off based on threat model and operational maturity."

Interview Question: How do you handle startup, baseline profiles, WebView, exported components, and mapping files?

Asked As / Variations

- Cold start vs warm start?
- What are baseline profiles?
- What is Macrobenchmark for?
- How do you secure WebView bridges?
- What are exported component risks?
- What are R8 keep rules?
- Why preserve mapping files?
- How do you handle crash spikes after rollout?

Strong Answer

"For startup, I separate cold, warm, and hot start because the bottlenecks can differ. Cold start includes process creation and app initialization, so I look for heavy work in `Application`, eager dependency graph creation, synchronous disk I/O, slow content providers, and too much work before first draw. Baseline Profiles can improve startup and runtime performance by helping Android precompile important code paths. Macrobenchmark is useful when I want repeatable startup or scroll performance measurements outside a normal unit test.

For WebView, I treat JavaScript bridges as a security boundary. I avoid exposing broad native APIs, restrict loaded content, validate origins where possible, disable unnecessary capabilities, and never assume web content is trusted just because it is inside my app.

Exported components are also entry points. An exported Activity, Service, or Receiver can be invoked by other apps if allowed by the manifest and permissions. I validate incoming intents, avoid privileged actions from untrusted inputs, and protect components with permissions or make them non-exported when external access is not needed.

For R8, keep rules preserve code needed by reflection, serialization, DI, or framework callbacks. I test release-like builds because debug builds do not prove minified behavior. Mapping files are essential because obfuscated crashes are much harder to diagnose without them.

If a rollout causes crash spikes, I stop or pause the rollout, compare release diffs and crash clusters, use mapping files to read stack traces, decide whether to hotfix or roll back, and add a regression test or release checklist item so the same issue is harder to ship again."

Tricky Follow-Up Questions And Answers

Follow-up: Can secrets be hidden in the APK?

Answer: "No, not reliably. Anything shipped to the client can be extracted. The server must enforce authorization; the app can only raise the cost of abuse."

Follow-up: What are exported component risks?

Answer: "Other apps may be able to start the component or send it data. If that component trusts intent extras or performs privileged actions, it can become an attack surface."

Follow-up: Why preserve mapping files?

Answer: "They let the team deobfuscate release crashes. Without them, production stack traces may not point to useful source names, which slows incident response."

Topic Drill Questions

Study these as interview prompts. First answer out loud, then compare with the senior answer and practice the follow-ups.

Question 1: How do you investigate jank?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 2: What causes ANRs?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 3: How do you investigate memory leaks?

Senior answer: "I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered:

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

Question 4: How do you improve startup performance?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 5: What tools do you use for performance?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 6: How do you store auth tokens?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 7: Can secrets be hidden in the APK?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 8: Certificate pinning: good idea or risk?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 9: How do you secure deep links?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 10: What are exported component risks?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 11: What can R8 break?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 12: How do you test release builds?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 13: What are mapping files?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 14: How do you handle crash spikes after rollout?

Senior answer: "After a crash spike, I first protect users: check rollout percentage, affected version, crash-free users, stack traces, device/API concentration, feature flags, and whether we should pause or roll back. Then I group crashes by root cause, deobfuscate with mapping files, reproduce if possible, and look for recent changes around the failing path. The fix should be small, reviewed, and released with monitoring. I would also add a regression test or guardrail when possible. The senior part is operational discipline: do not randomly patch symptoms, preserve evidence, communicate impact, and verify the spike actually drops after mitigation."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 15: What is cold start vs warm start?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 16: What are baseline profiles?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 17: What is Macrobenchmark for?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 18: What does LeakCanary detect?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 19: How do you secure WebView bridges?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 20: What are deep link security risks?

Senior answer: "I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered:

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Question 21: What are R8 keep rules?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Question 22: Why preserve mapping files?

Senior answer: "I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered:

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

10. Soft Skills

Study Anchors

There is no single official documentation source for behavioral interviews. Use your own project history and prepare stories using the framework below. Each story must be concrete enough to survive follow-up questions.

Theory To Know

Senior behavioral interviews test judgment, ownership, communication, mentorship, conflict resolution, and decision-making under ambiguity.

Prepare stories for:

- architecture disagreement
- mentoring
- production incident
- technical debt
- mistake/failure
- ambiguous requirements
- product/design/backend conflict
- project deep dive

Interview Question: Tell me about a time you disagreed with an architecture decision.

Strong Answer

"In one project, we debated whether to introduce a stricter MVI approach for a checkout flow. Some engineers wanted to use it across the whole feature because the state was getting complicated; others were worried it would slow delivery and make onboarding harder.

I tried to move the conversation away from preference. I wrote down what we were optimizing for: correctness of state transitions, testability, delivery time, and maintenance cost. Then I proposed a small prototype for the most complex part of the flow instead of changing the whole feature at once.

The result was that we used a reducer-style state model only where it paid off and kept simpler MVVM elsewhere. That gave us predictable state without turning the whole codebase into a pattern migration. The lesson was that architecture decisions should be scoped to the problem, not to the pattern someone likes."

Interview Question: Tell me about a mistake you made.

Strong Answer

"I once underestimated a local database migration because the schema change looked small. We caught an edge case late in QA where old data did not map correctly into the new model. It was not a production incident, but it was a clear miss in how I assessed risk.

I owned the mistake and changed the process. I added migration tests with representative old data, not just schema validation, and updated the release checklist so future database migrations had to include migration coverage before release candidates.

The lesson was that persistence changes need evidence, not confidence. With local data, once a migration runs on a user's device, a rollback may not fully undo the damage. Since then I treat database migrations as release-risk work even when the code diff is small."

Interview Question: Tell me about mentoring another developer.

Strong Answer

"A developer I worked with was struggling with coroutine scope ownership. They were fixing cancellation issues by adding new scopes, which made bugs disappear locally but created work that could outlive the screen.

I paired with them on one bug and focused on the mental model: the scope should match the lifetime of the work. We walked through why `viewModelScope` was right for screen-owned work, why WorkManager was better for persistent retryable work, and why swallowing cancellation could cause stale updates.

After that, I asked them to write a short team note and lead the next similar fix with me reviewing. The useful result was not just one bug fixed; they became able to spot the same pattern in reviews."

Interview Question: How do you handle conflict, ambiguity, debt, incidents, and AI tooling?

Asked As / Variations

- Tell me about a project you led.
- Tell me about a production incident.
- How do you communicate technical debt to product?
- How do you handle code review conflict?
- How do you lead without authority?
- Tell me about ambiguous requirements.
- How are you using AI tooling responsibly?

Strong Answer

"For behavioral questions, I avoid generic values like 'communication is important.' I give a concrete story with context, trade-off, action, result, and reflection.

For conflict, I try to move the conversation from preference to criteria: user impact, risk, maintainability, delivery time, reversibility, and evidence. In code review, I separate correctness issues from style preferences, and I try to propose a path forward instead of just blocking.

For ambiguity, I clarify the decision that must be made now versus what can stay flexible. I write assumptions down, identify reversible and irreversible decisions, and create a small validation step when possible.

For technical debt, I translate it into product risk: slower feature delivery, higher incident risk, broken tests, release delays, onboarding cost, or inability to support a roadmap item. Product teams respond better when debt is connected to outcomes.

For incidents, I focus on containment, communication, diagnosis, recovery, and prevention. I do not try to look perfect. A senior answer shows ownership and learning without blaming.

For AI tooling, my answer is balanced: I use it to speed up exploration, boilerplate, test ideas, debugging hypotheses, and documentation drafts, but I verify generated code, run tests, check security/privacy constraints, and do not let it bypass design judgment or code review."

Tricky Follow-Up Questions And Answers

Follow-up: How do you lead without authority?

Answer: "I create clarity: write the problem, propose options, make trade-offs visible, invite feedback, and help the group converge. Leadership is often reducing uncertainty, not giving orders."

Follow-up: What would your team say is hard about working with you?

Answer: "I would choose a real but controlled weakness. For example: I can push strongly for reliability when I see release risk, so I have learned to state the risk, propose options, and ask what level of risk the team wants to accept instead of sounding like there is only one acceptable path."

Follow-up: How do you answer without sounding memorized?

Answer: "I keep the story specific: project, constraint, decision, trade-off, result. If the interviewer asks follow-ups, I can explain details because the story is real rather than a script."

Topic Drill Questions

Study these as behavioral interview prompts. First answer out loud, then compare with the senior answer shape and practice the follow-ups.

Question 1: Tell me about a project you led.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 2: Tell me about an architecture disagreement.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 3: Tell me about a time you mentored someone.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 4: Tell me about a mistake you made.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 5: Tell me about a production incident.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 6: How do you communicate technical debt to product?

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 7: How do you handle code review conflict?

Senior answer: "I would handle a code review conflict by separating correctness, risk, and preference. If the comment is about correctness, security, lifecycle, or maintainability, I slow down and either fix it or explain the trade-off with evidence. If it is style or preference, I point to team conventions or propose a small consistent rule instead of debating taste. I try to move the discussion from personal opinion to code behavior: tests, complexity, ownership, rollout risk, and future maintenance. If we still disagree, I suggest a reversible decision, pair on the concern, or involve the owner of that area without turning the review into a status contest."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 8: How do you lead without authority?

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 9: Tell me about ambiguous requirements.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 10: What would your team say is hard about working with you?

Senior answer: "I would answer with a real edge, not a fake weakness. For example: I can push hard for clarity when ownership or failure modes are vague, and that can feel intense if I do not first align on urgency. Then I would show the mitigation: I now separate must-fix risks from preferences, write down trade-offs, ask whether the team needs a quick decision or deeper design, and invite disagreement earlier. The senior signal is self-awareness plus a changed behavior. I should not say 'I care too much' or blame others; I should show that my strength has a cost and that I manage that cost."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 11: Tell me about a time you pushed back on product.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 12: Tell me about technical debt you paid down.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 13: Tell me about a time you were wrong in a technical discussion.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 14: Tell me about a time you improved team process.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 15: Tell me about a time you had to make a reversible decision.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 16: Tell me about leading without authority.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 17: Tell me about a code review conflict.

Senior answer: "I would handle a code review conflict by separating correctness, risk, and preference. If the comment is about correctness, security, lifecycle, or maintainability, I slow down and either fix it or explain the trade-off with evidence. If it is style or preference, I point to team conventions or propose a small consistent rule instead of debating taste. I try to move the discussion from personal opinion to code behavior: tests, complexity, ownership, rollout risk, and future maintenance. If we still disagree, I suggest a reversible decision, pair on the concern, or involve the owner of that area without turning the review into a status contest."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question 18: Tell me about a time you handled ambiguity.

Senior answer: "I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered:

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Question Coverage

Question Coverage

> *Quality status: **Traceability audit**. Last audit: 2026-05-25.*

Purpose: verify that the practice questions are answerable from the documented study sections. If a question cannot be answered from `STUDY-GUIDE.md`, the guide is incomplete.

How To Use This File

- Pick a question from `QUESTION-BANK.md` or from a `Topic Drill Questions` section in `STUDY-GUIDE.md`.
- Study the mapped topic section.
- Answer out loud for 60-120 seconds.
- If the answer feels thin, mark it as `Needs expansion` and expand the topic section, not only the question bank.

Coverage Summary

Current exact traceability check:

- `QUESTION-BANK.md` numbered questions: **228**
- `STUDY-GUIDE.md` topic drill questions: **228**
- exact question-bank prompts missing from study guide: **0**
- study-guide drill questions with `Senior answer`: **228**
- study-guide drill questions with `Tricky follow-ups answered`: **228**
- study-guide tricky follow-up questions: **684**
- study-guide tricky follow-up answers: **684**
- mock interview tricky follow-up questions: **285**
- mock interview tricky follow-up answers: **285**

```
| Question Area | Bank Range | Study Section | Coverage | Notes |
|---|---:|---|---|---|
| Kotlin data classes, equality, null safety, sealed types, generics | 1-30, 153-162 | `STUDY-GUIDE.md` Part 1 | Direc
| Android lifecycle, saved state, context, intents, permissions, deep links | 31-45, 163-170 | `STUDY-GUIDE.md` Part 2
| Coroutines, cancellation, Flow, lifecycle collection, operators | 46-67, 171-180 | `STUDY-GUIDE.md` Part 3 | Direct
| Compose state, effects, recomposition, performance, testing | 68-82, 181-188 | `STUDY-GUIDE.md` Part 4 | Direct | Ex
| Architecture, Clean Architecture, DI, Hilt, modularization | 83-95, 189-196 | `STUDY-GUIDE.md` Part 5 | Direct | Exp
| Design patterns | 96-105 | `STUDY-GUIDE.md` Part 6 | Direct | Covered through problem-first answers: Repository, Obs
| Mobile system design | 106-118, 197-204 | `STUDY-GUIDE.md` Part 7 | Direct | Expanded offline-first, photo upload, t
| Testing | 119-128, 205-212 | `STUDY-GUIDE.md` Part 8 | Direct | Expanded `MainDispatcherRule`, `StandardTestDispatch
| Performance, security, release | 129-142, 213-220 | `STUDY-GUIDE.md` Part 9 | Direct | Expanded startup, baseline pr
| Soft skills | 143-152, 221-228 | `STUDY-GUIDE.md` Part 10 | Direct | Expanded leadership, debt, incidents, code revi
```

Forum-Driven Additions

These additions came from comparing the guide against public candidate/interviewer discussions. Forum sources are used for question phrasing and emphasis, while technical answers are anchored in official Kotlin and Android documentation.

```
| Source Pattern | Added Or Reinforced In Guide |
|---|---|
| Recent Android interview reports mention MVVM, multimodule setup, coroutines, Compose, DI, Hilt/Koin, testing with M
| Compose interview reports mention state hoisting, modifiers, recomposition, StateFlow, `LaunchedEffect`, `Disposable
| Senior interview advice emphasizes system design, backend/database/networking awareness, pitfalls, Clean Architectur
| Mobile system design discussions emphasize offline-first/caching, orientation/process death, functional scalability,
| Recent candidate feedback mentions identifying unnecessary work, unnecessary recomposition, uncancelled async work,
```

High-Risk Questions Now Covered

These were the questions most likely to expose gaps before this pass:

- Data class generated `equals`, number of property comparisons, `hashCode`, shallow `copy`, and mutable keys.
- Fragment view lifecycle and clearing binding.
- `PendingIntent`, deep links, task/back-stack behavior, and exported components.
- `callbackFlow` and why `awaitClose` is not optional.
- `combine` vs `zip`, `flatMapLatest` vs `mapLatest`, `debounce`, and one-shot parallel work with `async`.
- Compose stability, skippability, `derivedStateOf`, LazyColumn keys, snapshot state, and semantics testing.
- Hilt scopes, Hilt vs Dagger vs Koin, dispatcher qualifiers, Worker injection, and dependency replacement in tests.
- WorkManager testing, `MainDispatcherRule`, virtual time, Turbine, and testing `stateIn`.
- Baseline Profiles, Macrobenchmark, WebView bridges, exported component risks, R8 keep rules, and mapping files.
- Behavioral stories for debt, conflict, ambiguity, incidents, leadership without authority, and AI tooling.

Remaining Improvement Rule

Every future question added to `QUESTION-BANK.md` must be added under the matching `Topic Drill Questions` block in `STUDY-GUIDE.md` and mapped here. If the guide cannot answer it, expand the topic theory and strong answer before adding more questions.

Question Bank

> *Quality status: **Student practice draft**. Target: expand to 250-400 variations. This file is for drilling questions without reading full explanations.*

Use this after reading each topic in `STUDY-GUIDE.md`. Try answering out loud first, then check the topic chapter or study guide.

The same questions are now also grouped under each topic in `STUDY-GUIDE.md` as `Topic Drill Questions`, so you can study theory and practice immediately without jumping between files. This file remains the global drill bank for random review.

Answer Map

Use these sections when you get stuck:

- Questions 1-30: `STUDY-GUIDE.md` Part 1, Kotlin Fundamentals.
- Questions 31-45: `STUDY-GUIDE.md` Part 2, Android Fundamentals.
- Questions 46-67: `STUDY-GUIDE.md` Part 3, Coroutines And Flow.
- Questions 68-82: `STUDY-GUIDE.md` Part 4, Jetpack Compose.
- Questions 83-95: `STUDY-GUIDE.md` Part 5, Architecture.
- Questions 96-105: `STUDY-GUIDE.md` Part 6, Design Patterns.
- Questions 106-118: `STUDY-GUIDE.md` Part 7, Mobile System Design.
- Questions 119-128: `STUDY-GUIDE.md` Part 8, Testing.
- Questions 129-142: `STUDY-GUIDE.md` Part 9, Performance, Security, And Release.
- Questions 143-152: `STUDY-GUIDE.md` Part 10, Soft Skills.
- Questions 153-228: follow-up variations. Use the matching topic part in `STUDY-GUIDE.md`, then deepen with the topic chapter.

Study rule: do not read the answer first. Speak for 60-120 seconds, then check the guide.

Kotlin Fundamentals

Data Classes And Equality

- What is a Kotlin data class?
- What functions does a data class generate?
- Which properties are included in generated `equals` and `hashCode`?
- Are properties declared inside the class body included in `copy()`?
- What is the difference between `==` and `===`?
- How does generated `equals` work internally?
- How many comparisons can `equals` do for a data class with five properties?
- When is `hashCode` used?
- Why must `equals` and `hashCode` be consistent?
- What can go wrong if a mutable data class is used as a `HashMap` key?

- Is `copy()` deep or shallow?
- When should you avoid a data class?

Null Safety

- Kotlin is null-safe. Can it still throw `NullPointerException`?
- What is a platform type?
- When would you use `!!`?
- What is the difference between `String`, `String?`, and a Java platform type?
- What is the difference between `lateinit`, `lazy`, and nullable properties?
- Would you return `null` or an empty list from a repository?
- How do you model loading, empty, content, and error states?

Language Features

- What is a sealed class?
- Sealed class vs enum?
- What is an object declaration?
- Companion object vs object?
- What are extension functions?
- Can extension functions override member functions?
- What are Kotlin scope functions and when can they hurt readability?
- What does `inline` do?
- Why does `reified` require `inline`?
- What is type erasure?
- Explain `in` and `out` in Kotlin generics.

Android Fundamentals

- Walk me through Activity lifecycle.
- Walk me through Fragment lifecycle.
- What survives configuration change?
- What survives process death?
- Does ViewModel survive process death?
- What belongs in `SavedStateHandle`?
- What should go into Room or DataStore instead of saved state?
- Why should ViewModel not hold an Activity or View reference?
- Application context vs Activity context?
- What causes memory leaks in Android?
- How do you handle runtime permissions?

- What is an intent?
- Explicit vs implicit intent?
- What are deep links and what can go wrong with them?
- BroadcastReceiver vs Service vs WorkManager?

Coroutines And Flow

- What is a coroutine?
- Are coroutines threads?
- Does `suspend` mean background thread?
- What is structured concurrency?
- `launch` vs `async` vs `withContext`?
- What does `async` return?
- What happens if you call `async` and never `await`?
- What happens when a child coroutine fails?
- `coroutineScope` vs `supervisorScope`?
- What is `CancellationException`?
- Why should you not swallow cancellation?
- Why is `GlobalScope` discouraged?
- `Dispatchers.IO` vs `Dispatchers.Default`?
- What is Flow?
- Cold Flow vs hot Flow?
- Flow vs StateFlow vs SharedFlow?
- How do you model one-off events?
- What does `flowOn` affect?
- What does `catch` catch?
- `collect` vs `collectLatest`?
- `stateIn` vs `shareIn`?
- How do you collect Flow safely in Android?

Jetpack Compose

- What is recomposition?
- Does recomposition redraw the whole screen?
- What triggers recomposition?
- What is state hoisting?
- Does all state belong in ViewModel?
- `remember` vs `rememberSaveable`?

- What should survive process death in Compose?
- What is `LaunchedEffect`?
- When does `LaunchedEffect` restart?
- What is `DisposableEffect`?
- What is `rememberUpdatedState` for?
- Why can mutating a list not update Compose UI?
- What are lazy list keys?
- How do you handle navigation events in Compose?
- How do you investigate Compose performance?

Architecture

- Explain MVVM in Android.
- MVVM vs MVI?
- What is Clean Architecture?
- When is Clean Architecture overkill?
- What is the Repository pattern?
- What should not go into a Repository?
- When do you use UseCases?
- What is unidirectional data flow?
- What is single source of truth?
- DTO vs domain model vs UI model?
- How do you avoid ViewModel becoming too large?
- How would you migrate a legacy MVP/XML app?
- How would you modularize a large Android app?

Design Patterns

- What design patterns have you used in Android?
- Singleton: when is it okay and when is it dangerous?
- Factory vs dependency injection?
- Adapter pattern examples in Android?
- Observer pattern in Android?
- Strategy pattern example in Android?
- State pattern example in Android?
- Command pattern for offline operations?
- How do Kotlin features reduce the need for some classic patterns?
- What is pattern abuse?

Mobile System Design

- Design an offline-first feed.
- Design an offline notes app.
- Design a chat feature.
- Design photo upload with retry.
- Design background sync.
- WorkManager vs foreground service?
- How do you handle offline writes?
- How do you handle sync conflicts?
- How do you avoid duplicate writes?
- How do you handle logout with pending offline work?
- Design token refresh architecture.
- Design app startup for a large app.
- Design feature flags for mobile.

Testing

- How do you test a ViewModel with coroutines?
- How do you test Flow emissions?
- How do you test StateFlow initial state?
- How do you avoid real delays in tests?
- What is dispatcher injection?
- Fakes vs mocks?
- How do you test Compose UI?
- How do you test navigation behavior?
- How do you test Room migrations?
- How do you reduce flaky UI tests in CI?

Performance, Security, Release

- How do you investigate jank?
- What causes ANRs?
- How do you investigate memory leaks?
- How do you improve startup performance?
- What tools do you use for performance?
- How do you store auth tokens?
- Can secrets be hidden in the APK?
- Certificate pinning: good idea or risk?

- How do you secure deep links?
- What are exported component risks?
- What can R8 break?
- How do you test release builds?
- What are mapping files?
- How do you handle crash spikes after rollout?

Soft Skills

- Tell me about a project you led.
- Tell me about an architecture disagreement.
- Tell me about a time you mentored someone.
- Tell me about a mistake you made.
- Tell me about a production incident.
- How do you communicate technical debt to product?
- How do you handle code review conflict?
- How do you lead without authority?
- Tell me about ambiguous requirements.
- What would your team say is hard about working with you?

More Follow-Up Variations

- How would you explain a data class without using the word "boilerplate"?
- What happens if a data class contains a mutable list?
- How do data classes interact with DiffUtil or Compose state comparisons?
- Why can `lateinit` be dangerous?
- When is `lazy` initialized?
- How would you model an API result in Kotlin?
- What is the difference between `Result`, sealed classes, and exceptions?
- Can a sealed class be extended outside its package/module?
- What is a value class?
- When would a value class help in domain modeling?
- What is the difference between `onCreate`, `onStart`, and `onResume`?
- Fragment view lifecycle vs Fragment lifecycle?
- Why should Fragment binding be cleared?
- What is a PendingIntent?
- What is task/back-stack behavior for deep links?
- What can go wrong with saving too much state in a Bundle?

- How do you handle background location permission?
- What are exported components?
- What dispatcher should CPU-heavy work use?
- What dispatcher should blocking I/O use?
- What happens if you block `Dispatchers.Main`?
- How do you run two requests in parallel and combine results?
- What is `NonCancellable` used for?
- What is `callbackFlow`?
- Why is `awaitClose` important?
- What is `debounce` useful for?
- `combine` vs `zip`?
- `flatMapLatest` vs `mapLatest`?
- What makes a type stable in Compose?
- What is skippability?
- What is `derivedStateOf` for?
- When can `derivedStateOf` be overused?
- How do LazyColumn keys help?
- How do you avoid repeated navigation in Compose?
- How do you test Compose semantics?
- What is snapshot state?
- Hilt vs Dagger?
- Hilt vs Koin?
- What are Hilt scopes?
- What should be singleton scoped?
- Why qualify dispatchers in DI?
- How do you inject workers?
- How do you replace dependencies in tests?
- What is dependency inversion?
- Design offline-first notes with conflict resolution.
- Design chat message sending with retry.
- Design pagination with cache invalidation.
- Design feature flags for Android.
- Design app startup initialization.
- How do you handle user logout in an offline-first app?
- How do you clean pending work after logout?
- How do you monitor sync failures?

- What is `MainDispatcherRule`?
- `StandardTestDispatcher` vs `UnconfinedTestDispatcher`?
- What does `advanceUntilIdle()` do?
- How do you test retry with virtual time?
- What is Turbine used for?
- How do you test `stateIn`?
- How do you test WorkManager?
- What makes UI tests flaky?
- What is cold start vs warm start?
- What are baseline profiles?
- What is Macrobenchmark for?
- What does LeakCanary detect?
- How do you secure WebView bridges?
- What are deep link security risks?
- What are R8 keep rules?
- Why preserve mapping files?
- Tell me about leading without authority.
- Tell me about a code review conflict.
- Tell me about a time you pushed back on product.
- Tell me about technical debt you paid down.
- Tell me about a time you were wrong in a technical discussion.
- Tell me about a time you improved team process.
- Tell me about a time you handled ambiguity.
- Tell me about a time you had to make a reversible decision.

Mock Interviews

> *Quality status: **Interview-ready practice draft**. Each question includes an interview-friendly answer and what a senior should demonstrate.*

Use these as spoken practice. Set a timer, answer out loud first, then compare with the answer below the question. Do not memorize the exact wording. Study the shape: direct answer, Android implication, trade-off, and senior judgment.

Scoring Rubric

Score each answer from 0 to 4:

- **0**: I could not answer.
- **1**: I knew the term but gave a shallow definition.
- **2**: I gave a mostly correct answer but missed follow-ups or edge cases.
- **3**: I gave a clear answer with one trade-off or Android-specific implication.
- **4**: I sounded senior: clear, practical, technically precise, and ready for follow-ups.

After each round, mark:

- weakest topic,
- strongest topic,
- one answer that sounded memorized,
- one answer that sounded natural,
- one follow-up I could not handle.

Pass threshold:

- 70% for first pass,
- 80% for solid readiness,
- 90% for interview-ready confidence.

Round 1: Kotlin Fundamentals

Time: 35 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 1.
- `QUESTION-BANK.md` questions 1-30 and 153-162.

Score target: 32/40 or higher.

1. What is a data class in Kotlin?

Senior answer

"I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

2. What methods does a data class generate?

Senior answer

"I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

3. How does generated `equals` work?

Senior answer

"For a data class, generated `equals` is ordinary structural equality over the primary-constructor properties. Conceptually it checks quick cases first, such as same reference and compatible type, then compares each primary-constructor property in order using that property's equality. If the class has five primary-constructor properties, it can compare up to five properties after the cheap checks, but it can stop earlier when the first property differs. Body properties are not part of that generated equality. I would also connect this to `==`: in Kotlin, `==` calls `equals`, while `===` asks whether both references point to the same object."

Tricky follow-ups answered

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

4. When is `hashCode` used?

Senior answer

"`hashCode` is used by hash-based collections such as `HashMap` and `HashSet` to narrow where an object might live. The collection uses the hash to find a bucket and then uses `equals` to confirm the actual match, because different objects can share a hash. The contract is: if two objects are equal, they must have the same hash code. The reverse is not guaranteed. For data classes, generated `hashCode` uses the same primary-constructor properties as generated `equals`, so mutating those properties after insertion into a hash collection can make the object hard or impossible to find through normal lookup."

Tricky follow-ups answered

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

5. Kotlin is null-safe. How can NPE still happen?

Senior answer

"I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

6. What is a platform type?

Senior answer

"I would explain Kotlin null-safety as a type-system tool, not a magic shield. `String` and `String?` are different contracts, Java platform types can still surprise you, `!!` converts uncertainty into a possible crash, and `lateinit` fails at runtime if read before initialization. In senior code I prefer explicit modeling: nullable only when absence is meaningful, empty collections when the result is valid but empty, and sealed/result types when I need loading, error, or permission states. The answer should name the remaining NPE paths and show how I keep nullability at boundaries instead of spreading defensive checks everywhere."

Tricky follow-ups answered

Follow-up: Where can NPE still come from?

Answer: Platform types, `!!`, `lateinit` before initialization, Java interop, reflection/serialization, and framework callbacks can still create runtime null failures.

Follow-up: When is `null` the right model?

Answer: When absence is a real domain state. If the result is successfully empty, prefer an empty collection. If it is loading/error, model that explicitly.

Follow-up: Why is `!!` risky?

Answer: It moves uncertainty from the type system into a runtime crash. It should be rare and backed by an invariant you can explain.

Follow-up: How do you keep nullability clean?

Answer: Validate at boundaries, map platform types into Kotlin contracts, and avoid spreading nullable state deeper than necessary.

7. Sealed class vs enum?

Senior answer

"A sealed class represents a closed hierarchy: the compiler knows the permitted subtypes, so `when` expressions can be exhaustive without an `else` when every case is handled. I use it when alternatives carry different data or behavior, such as `Loading`, `Content(items)`, `Empty`, and `Error(cause)`. An enum is better for a fixed list of constants with the same shape, such as sort order or theme mode. The senior detail is choosing the model that makes impossible states impossible: sealed hierarchies are great for typed UI state and domain results, but they can be overkill for simple constant sets."

Tricky follow-ups answered

Follow-up: When is sealed better than enum?

Answer: Use sealed when cases carry different data or behavior and you want exhaustive handling. Use enum for fixed same-shaped constants.

Follow-up: What bug does a state model prevent?

Answer: It prevents impossible combinations such as loading plus content plus error unless that combination is intentionally represented.

Follow-up: What should the `when` expression show?

Answer: It should handle every state explicitly, ideally exhaustively, so adding a new state forces compile-time review.

Follow-up: When is this overkill?

Answer: When a simple Boolean, nullable field, or enum fully captures the domain without ambiguous combinations.

8. Why does `reified` require `inline`?

Senior answer

"I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

9. Explain `in` and `out` in generics.

Senior answer

"I would connect the language feature to runtime behavior. JVM generics are erased, so `reified` only works with `inline` because the compiler copies the function body at call sites and can substitute the real type token. Variance controls safe substitution: `out` is for producers I read from, `in` is for consumers I write into. Value classes can make domain IDs and small wrappers more type-safe with less allocation in many cases, but they still have boxing edges. The senior angle is not naming features; it is knowing when they make APIs safer and when they make code clever without improving the model."

Tricky follow-ups answered

Follow-up: What is the runtime detail?

Answer: Generic type information is erased on the JVM. `inline` plus `reified` lets the compiler substitute type information at call sites.

Follow-up: How do `in` and `out` map to use?

Answer: `out` is for producers you read from; `in` is for consumers you write to. The goal is safe substitution.

Follow-up: Where can value classes surprise you?

Answer: They can box at generic/interface/nullability boundaries, so they improve type safety but are not magic performance tools.

Follow-up: How do you avoid sounding theoretical?

Answer: Tie the feature to safer API design, fewer invalid IDs, better typed boundaries, or avoiding unsafe casts.

10. When can scope functions make code worse?

Senior answer

"I would describe how Kotlin resolves the construct and what bug it can create. `object` creates a singleton, a companion object holds class-associated members, and extension functions are statically resolved by the declared receiver type. That means extensions do not truly override members; member functions win. Scope functions are useful for local object configuration or transformations, but they hurt readability when nested or when `it`/`this` hides ownership. In senior Android code I care less about using every Kotlin feature and more about whether the feature clarifies lifetime, dependency ownership, API shape, and testability."

Tricky follow-ups answered

Follow-up: What is statically resolved?

Answer: Extension functions are resolved by the declared receiver type and do not override member functions.

Follow-up: When is `object` dangerous?

Answer: When it hides global mutable state, makes tests order-dependent, or owns Android resources with unclear lifetime.

Follow-up: When do scope functions hurt?

Answer: When nested calls hide which receiver is being used or when `it`/`this` obscures side effects.

Follow-up: How do you decide whether to use it?

Answer: Use the feature only when it clarifies construction, ownership, or call-site readability.

Round 2: Android Fundamentals

Time: 35 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 2.
- `QUESTION-BANK.md` questions 31-45 and 163-170.

Score target: 32/40 or higher.

1. Explain Activity lifecycle.

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

2. Explain Fragment lifecycle.

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

3. Rotation vs process death?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

4. What survives process death?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

5. What belongs in `SavedStateHandle`?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

6. Why should ViewModel not hold a View reference?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

7. Activity context vs application context?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

8. How do Android memory leaks usually happen?

Senior answer

"Android leaks usually happen when an object with a longer lifetime holds an object with a shorter lifetime. Common examples are a singleton holding an Activity context, a ViewModel holding a View or Fragment binding, a callback not being unregistered, a coroutine outliving the UI scope, or Fragment view binding surviving after `onDestroyView`. I would explain it as a lifetime mismatch, not just 'forgot to clear something'. The fix is to put work in the correct scope, use application context only for app-lifetime needs, clear view references at the view lifecycle boundary, unregister listeners, and verify suspicious cases with tools like LeakCanary."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

9. What are deep links?

Senior answer

"I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

10. BroadcastReceiver vs Service vs WorkManager?

Senior answer

"I would treat Android entry points as lifecycle and trust-boundary problems. Intents, deep links, permissions, PendingIntents, broadcasts, services, WorkManager, and exported components can be triggered by the system, another app, a notification, a cold start, or restored state. I validate extras, IDs, auth/session state, URI ownership, and destination before doing privileged work. For background work I choose based on guarantee and visibility: WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers only for short event handling. The senior answer includes cold-start behavior, OS limits, security, and user-visible recovery."

Tricky follow-ups answered

Follow-up: What must be validated?

Answer: Intent extras, URI parameters, auth/session state, permissions, exported status, and destination authorization.

Follow-up: How do you choose background work?

Answer: Use WorkManager for deferrable persistent work, foreground service for user-visible ongoing work, and receivers for short event handling.

Follow-up: What is the cold-start problem?

Answer: A deep link or notification may enter the app without previous in-memory navigation state, so the destination must rebuild required context.

Follow-up: What is the security angle?

Answer: Treat external entry points as untrusted input and avoid exposing privileged actions through exported components.

Round 3: Coroutines And Flow

Time: 45 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 3.
- `QUESTION-BANK.md` questions 46-67 and 171-180.

Score target: 38/48 or higher.

1. What is a coroutine?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

2. Are coroutines threads?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

3. Does `suspend` mean background?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

4. `launch` vs `async` vs `withContext`?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

5. What happens when one child coroutine fails?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

6. What is `supervisorScope`?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

7. Why should `CancellationException` usually propagate?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

8. Flow vs StateFlow vs SharedFlow?

Senior answer

"I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

9. What does `flowOn` affect?

Senior answer

"I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

10. What does `catch` catch?

Senior answer

"In Flow, `catch` catches exceptions from upstream of where the operator is placed. It does not catch exceptions thrown downstream by the collector, and it should not be used to hide cancellation. A senior answer names placement: `source.map { ... }.catch { ... }.collect { ... }` catches failures from `source` and `map`, but not failures inside `collect`. If I recover, I emit a fallback state or map the failure into a typed error. If the failure is `CancellationException`, I normally let it propagate because cancellation is part of structured concurrency, not an ordinary business error."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

11. How do you collect Flow safely in Android?

Senior answer

"I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

12. How would you model one-off UI events?

Senior answer

"I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

Round 4: Jetpack Compose

Time: 40 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 4.
- `QUESTION-BANK.md` questions 68-82 and 181-188.

Score target: 32/40 or higher.

1. What is recomposition?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

2. Does recomposition redraw the whole screen?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

3. What is state hoisting?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

4. Does all Compose state belong in ViewModel?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

5. `remember` vs `rememberSaveable`?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

6. What is `LaunchedEffect`?

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

7. When does an effect restart?

Senior answer

"I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

8. Why can mutating a list not update UI?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

9. How do you handle navigation events?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

10. How do you investigate Compose performance?

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

Round 5: Architecture

Time: 45 minutes

Study before this round:

- `STUDY-GUIDE.md` Parts 5 and 6.
- `QUESTION-BANK.md` questions 83-105 and 189-196.

Score target: 32/40 or higher.

1. Explain MVVM in Android.

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

2. MVVM vs MVI?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

3. What is Clean Architecture?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

4. When is Clean Architecture overkill?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

5. What is Repository responsible for?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

6. When do you use UseCases?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

7. What is single source of truth?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

8. How do you avoid ViewModel bloat?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

9. How would you modularize a large app?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

10. How would you migrate a legacy feature?

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

Round 6: Mobile System Design

Time: 60 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 7.
- `QUESTION-BANK.md` questions 106-118 and 197-204.

Score target: 36/44 or higher.

Prompt: Design an offline-first feed.

Senior answer

"I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Follow-up 1. What is the local source of truth?

Senior answer

"The local source of truth is the durable place the UI observes as authoritative for that feature, usually Room for relational/queryable app data or DataStore for small preferences. The repository coordinates network, cache freshness, mapping, and sync, but the UI should not juggle separate network and database truths. For an offline-first feature, writes are persisted locally with sync metadata, the UI reflects pending/synced/failed/conflicted states, and workers reconcile with the server. I would call it source of truth only if it survives process death and can recover after restart; an in-memory list inside a ViewModel is screen state, not the feature's durable truth."

Tricky follow-ups answered

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Follow-up 2. How do you handle offline writes?

Senior answer

"I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Follow-up 3. How do you handle conflicts?

Senior answer

"I would first ask what a conflict means for the product, because the right policy is domain-specific. Some data can use last-write-wins, some should be server-authoritative, some can merge field by field, and some must ask the user. Technically, I persist enough metadata to detect conflict: operation IDs, local version, remote version, updated timestamps when they are meaningful, and server response state. The UI should expose conflicted or failed states instead of silently dropping work. A senior answer also mentions idempotency, retries, monitoring conflict rate, and a recovery path after process death or logout."

Tricky follow-ups answered

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

Follow-up 4. How do you test it?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Round 7: Testing

Time: 35 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 8.
- `QUESTION-BANK.md` questions 119-128 and 205-212.

Score target: 26/32 or higher.

1. How do you test ViewModel state?

Senior answer

"I test ViewModel state by driving the public inputs and asserting the emitted state sequence. If the ViewModel uses coroutines or Flow, I run the test with `runTest`, replace Main with a test dispatcher rule, inject dispatchers instead of hardcoding them, and use fakes for repositories so the test controls data and errors. I assert the initial state, loading state when relevant, success, empty, and failure paths. I avoid testing private functions or implementation timing. The important senior detail is determinism: no real network, no real delays, no uncontrolled dispatcher, and no assertion that depends on scheduler luck."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

2. How do you test coroutines?

Senior answer

"I test coroutine code with `runTest` so delays use virtual time and child coroutines are tracked by the test scope. I inject dispatchers or a dispatcher provider so production code does not hardcode Main, IO, or Default in a way the test cannot control. For code that launches work, I advance the scheduler and assert observable state, returned results, or side effects through fakes. I also test cancellation and error paths, not just success. A good interview answer says I avoid `Thread.sleep`, avoid real dispatchers in unit tests, and make structured concurrency visible to the test instead of hiding work in global scopes."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

3. How do you test Flow emissions?

Senior answer

"I test Flow by collecting it in a controlled coroutine test and asserting emissions in order. Turbine is useful because it lets me `awaitItem`, assert completion or errors, and check that no unexpected events arrived. For `StateFlow`, I remember there is always an initial value, so the test should account for that before later emissions. For never-ending flows, I cancel the collection or use Turbine's cancellation helpers so the test does not hang. The senior detail is operator and lifecycle awareness: I control upstream fakes, virtual time for debounce/retry, and I assert behavior rather than the internal chain of operators."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

4. How do you handle never-ending flows in tests?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

5. Fakes vs mocks?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

6. How do you test Compose UI?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

7. How do you test Room migrations?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

8. How do you reduce flaky tests?

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

Round 8: Performance, Security, Release

Time: 45 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 9.
- `QUESTION-BANK.md` questions 129-142 and 213-220.

Score target: 32/40 or higher.

1. How do you investigate jank?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

2. What causes ANRs?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

3. How do you find memory leaks?

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

4. How do you improve startup?

Senior answer

"I would answer by naming the concept, the owner, the lifecycle boundary, the failure mode, and the trade-off. A senior Android answer should not stop at a definition. It should say what I would normally choose, when I would choose differently, and what bug the wrong choice creates. I would also mention how I would verify the behavior: unit test, integration test, profiler, release monitoring, or production metric depending on the risk. That makes the answer useful for interview study because it connects theory to the decisions an interviewer is usually probing."

Tricky follow-ups answered

Follow-up: What is the hidden failure mode?

Answer: Usually ownership, lifecycle, cancellation, invalid state, stale data, test nondeterminism, or production recovery.

Follow-up: What changes the answer?

Answer: Lifetime, risk, product guarantee, team convention, performance, security, and testability.

Follow-up: How would you verify it?

Answer: Use the smallest reliable signal: unit test, integration test, profiler, logs, metrics, or rollout monitoring.

Follow-up: What should you avoid?

Answer: Avoid absolute rules without context. Name the default, the exception, and why the trade-off matters.

5. How do you store tokens?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

6. Can secrets be hidden in the APK?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

7. Certificate pinning: when and why?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

8. What can R8 break?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

9. How do you test release builds?

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

10. What do you do after a crash spike?

Senior answer

"After a crash spike, I first protect users: check rollout percentage, affected version, crash-free users, stack traces, device/API concentration, feature flags, and whether we should pause or roll back. Then I group crashes by root cause, deobfuscate with mapping files, reproduce if possible, and look for recent changes around the failing path. The fix should be small, reviewed, and released with monitoring. I would also add a regression test or guardrail when possible. The senior part is operational discipline: do not randomly patch symptoms, preserve evidence, communicate impact, and verify the spike actually drops after mitigation."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

Round 9: Behavioral

Time: 45 minutes

Study before this round:

- `STUDY-GUIDE.md` Part 10.
- `QUESTION-BANK.md` questions 143-152 and 221-228.

Score target: 32/40 or higher.

1. Tell me about a project you led.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

2. Tell me about an architecture disagreement.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

3. Tell me about mentoring someone.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

4. Tell me about a mistake.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

5. Tell me about a production incident.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

6. How do you communicate technical debt?

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

7. How do you handle code review conflict?

Senior answer

"I would handle a code review conflict by separating correctness, risk, and preference. If the comment is about correctness, security, lifecycle, or maintainability, I slow down and either fix it or explain the trade-off with evidence. If it is style or preference, I point to team conventions or propose a small consistent rule instead of debating taste. I try to move the discussion from personal opinion to code behavior: tests, complexity, ownership, rollout risk, and future maintenance. If we still disagree, I suggest a reversible decision, pair on the concern, or involve the owner of that area without turning the review into a status contest."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

8. How do you work with product and design?

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

9. Tell me about ambiguous requirements.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

10. What would your team say is hard about working with you?

Senior answer

"I would answer with a real edge, not a fake weakness. For example: I can push hard for clarity when ownership or failure modes are vague, and that can feel intense if I do not first align on urgency. Then I would show the mitigation: I now separate must-fix risks from preferences, write down trade-offs, ask whether the team needs a quick decision or deeper design, and invite disagreement earlier. The senior signal is self-awareness plus a changed behavior. I should not say 'I care too much' or blame others; I should show that my strength has a cost and that I manage that cost."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Round 10: Mixed Senior Round

Time: 60 minutes

Score target: 34/40 or higher.

1. Kotlin data class internals.

Senior answer

"I would anchor the answer in Kotlin's value semantics. Data classes generate `equals`, `hashCode`, `toString`, `copy`, and `componentN` functions from primary-constructor properties only. `==` delegates to `equals`, while `===` is reference identity. Generated equality compares those constructor properties, and generated hash code must stay consistent with equality. The practical risk is mutability: `copy()` is shallow, nested mutable objects are shared, and changing a property used by hash code after insertion into a `HashMap` or `HashSet` can break lookup. So I use data classes for stable values, DTOs, UI state, and simple domain models, not for identity-heavy mutable objects."

Tricky follow-ups answered

Follow-up: What is the hidden edge case?

Answer: Generated methods use only primary-constructor properties. Body properties can differ while instances still compare equal, and `copy()` will not copy body properties through parameters.

Follow-up: How does this fail in collections?

Answer: Hash collections use `hashCode` to find a bucket and `equals` to confirm. If a key's hash-relevant state mutates, lookup and removal can fail.

Follow-up: What should you say about `copy()`?

Answer: `copy()` is shallow. The outer instance is new, but nested mutable objects may still be shared.

Follow-up: How would you avoid this bug?

Answer: Use immutable key fields, avoid mutable data classes as hash keys, or base equality/hash code only on stable identity.

2. Process death and state restoration.

Senior answer

"I would answer in terms of lifetime ownership. Activities, Fragments, Fragment views, ViewModels, saved state, and durable storage all survive different things. ViewModel can survive configuration change, but not process death. `SavedStateHandle` and saved instance state are for small restoration keys and UI inputs, while Room/DataStore handle durable data. Fragment view references die at `onDestroyView`, even if the Fragment instance remains. Most leaks are lifetime mismatches: long-lived objects holding Activity, View, binding, callbacks, or coroutines. A senior answer names what survives rotation, what survives process death, and which owner should clean up."

Tricky follow-ups answered

Follow-up: What survives rotation?

Answer: ViewModel can survive configuration change; Activity/Fragment views are recreated, and saved instance state can restore small UI state.

Follow-up: What survives process death?

Answer: Durable persistence such as Room/DataStore and saved-state snapshots can survive. In-memory singletons and ViewModels do not.

Follow-up: Where do leaks usually come from?

Answer: Long-lived objects retaining shorter-lived Activity, View, binding, callback, context, or coroutine references.

Follow-up: How do you decide the right owner?

Answer: Use the shortest owner that can safely hold the state, then move only durable or cross-screen data to longer-lived storage.

3. Coroutines cancellation scenario.

Senior answer

"I would explain coroutine ownership before syntax. A coroutine is a cancellable unit of async work running in a `CoroutineScope`; it is not the same thing as a thread. `suspend` means the function can suspend without blocking, but it does not automatically switch dispatchers. Structured concurrency means child work is tied to a parent lifetime, so cancellation and failure propagate predictably unless a supervisor boundary is used. `launch` returns `Job`, `async` returns `Deferred`, and `withContext` switches context for a result. I avoid `GlobalScope`, avoid blocking Main, and let `CancellationException` propagate."

Tricky follow-ups answered

Follow-up: Does `suspend` switch threads?

Answer: No. It allows suspension. Dispatcher choice or `withContext` decides where work runs.

Follow-up: What happens on child failure?

Answer: In a regular scope, failure usually cancels siblings and propagates to the parent. Supervisor boundaries isolate sibling failure.

Follow-up: Why not swallow cancellation?

Answer: `CancellationException` is the cooperative cancellation signal. Swallowing it can keep cancelled work alive and break structured concurrency.

Follow-up: What should you avoid?

Answer: Avoid `GlobalScope`, blocking Main, fire-and-forget `async`, and broad catches that hide cancellation or ownership.

4. Flow event modeling.

Senior answer

"I would start by saying Flow is an asynchronous stream with backpressure through suspension. Cold flows run per collector; hot flows exist independently of a collector. `StateFlow` represents current state with a latest value, while `SharedFlow` is for shared emissions and can be configured for replay. Operator placement matters: `flowOn` changes upstream context, `catch` catches upstream exceptions, and `collectLatest` cancels the previous collector block when a new value arrives. In Android, I collect with lifecycle awareness and model one-off events deliberately so navigation or snackbars do not replay after rotation."

Tricky follow-ups answered

Follow-up: Cold or hot?

Answer: Cold Flow starts per collector. StateFlow and SharedFlow are hot and can exist independently of a collector.

Follow-up: What does operator placement change?

Answer: `flowOn` affects upstream context; `catch` catches upstream failures; downstream collector failures are not caught by an upstream `catch`.

Follow-up: How do you collect safely in Android?

Answer: Use lifecycle-aware collection such as `repeatOnLifecycle` or Compose lifecycle-aware state collection.

Follow-up: How do you avoid event replay bugs?

Answer: Model events deliberately, choose replay behavior explicitly, and make one-off navigation/snackbar behavior lifecycle-aware.

5. Compose recomposition bug.

Senior answer

"I would frame Compose as state-driven UI. Recomposition reruns composable functions that read changed state; it does not mean the whole screen is redrawn. State should be hoisted to the owner that can make decisions: ViewModel for screen/business state, local `remember` for ephemeral UI state, and `rememberSaveable` for small values that should survive recreation. Effects like `LaunchedEffect` restart when keys change, so keys must represent the lifetime of the side effect. Performance answers should mention stable models, keys for lazy lists, avoiding mutable collections that Compose cannot observe, and measuring before optimizing."

Tricky follow-ups answered

Follow-up: What triggers recomposition?

Answer: Reads of snapshot state changing can invalidate composables that read that state.

Follow-up: What should be hoisted?

Answer: Hoist state to the lowest owner that needs to read or change it; use ViewModel for screen/business state and local state for ephemeral UI.

Follow-up: Why can mutable lists fail?

Answer: Mutating a normal list in place may not change observable state, so Compose may not know to recompose.

Follow-up: What do you measure?

Answer: Use recomposition tools, tracing, Macrobenchmark, frame timing, and targeted profiling before optimizing.

6. Repository/source of truth design.

Senior answer

"I would answer with ownership boundaries, not architecture buzzwords. UI renders state and emits events. ViewModel owns screen state and user-intent handling. Repositories own data policy: network, database, cache, freshness, sync, and mapping. Data sources own framework or service details. Use cases are useful when a business operation is reused, complex, or deserves a named boundary. DI owns construction and lifetime. Clean Architecture, MVVM, and modularization are tools to control dependency direction and change, but each layer must earn its place. A senior answer names the boundary, the failure it prevents, and when a simpler design is better."

Tricky follow-ups answered

Follow-up: Who owns what?

Answer: UI renders, ViewModel owns screen state, repositories own data policy, data sources own framework/API details, and DI owns construction.

Follow-up: When is a layer overkill?

Answer: When it only forwards calls without protecting a boundary, rule, test seam, or expected change.

Follow-up: What makes it testable?

Answer: Dependency inversion, injected dispatchers/services, deterministic state transitions, and boundaries that can be replaced with fakes.

Follow-up: What trade-off should you name?

Answer: Complexity, team size, feature volatility, release risk, module boundaries, and migration cost.

7. Offline write sync design.

Senior answer

"I would design for mobile failure first: flaky network, process death, auth changes, offline use, retries, duplicate submissions, battery, and OS background limits. A durable local source of truth gives UI a stable model. Pending operations need IDs, status, retry policy, idempotency keys, and reconciliation rules. WorkManager handles deferrable persistent background work; foreground service is for user-visible ongoing work. Conflicts require a product policy, not just code. A senior answer includes logout behavior, token refresh, cache invalidation, monitoring, and how the system recovers after restart without losing or duplicating user work."

Tricky follow-ups answered

Follow-up: What is the durable truth?

Answer: Usually Room or another persistent store observed by UI, with repositories/workers reconciling network state into it.

Follow-up: How do retries stay safe?

Answer: Persist operations with stable IDs, idempotency keys, retry policy, and terminal failure states.

Follow-up: How do you handle conflicts?

Answer: Choose a product policy: server wins, client wins, last-write-wins, merge, or user resolution.

Follow-up: What do you monitor?

Answer: Queue age, retry count, conflict rate, auth failures, duplicate attempts, terminal failures, and crash/error spikes.

8. ViewModel test strategy.

Senior answer

"I would emphasize deterministic behavior. Coroutine tests should use `runTest`, injected dispatchers, a Main dispatcher rule when needed, and virtual time instead of sleeps. Flow tests should assert emissions, completion, errors, and absence of extra events; Turbine is useful for that. ViewModel tests should verify state transitions through public inputs, not internal implementation. Compose tests should use semantics and avoid timing assumptions. Room migrations need real schema migration checks, and WorkManager should use its test helpers. The senior answer says what is controlled: time, dispatchers, dependencies, lifecycle, data, and external services."

Tricky follow-ups answered

Follow-up: What must the test control?

Answer: Dispatchers, time, dependencies, lifecycle, data, permissions, network, and external services.

Follow-up: Why avoid sleeps?

Answer: Sleeps make tests slow and flaky. Virtual time and explicit scheduler advancement make timing deterministic.

Follow-up: When are fakes better than mocks?

Answer: When behavior and state matter across multiple calls. Mocks are useful for narrow interaction checks.

Follow-up: Which failure paths should be covered?

Answer: Cover errors, cancellation, retries, empty states, race-prone lifecycle changes, and release-sensitive paths, not only happy cases.

9. Release crash spike.

Senior answer

"I would start with evidence and threat model. For performance, measure frame timing, main-thread work, startup path, allocations, I/O, and traces using Perfetto, Android Studio Profiler, Macrobenchmark, Baseline Profiles, Android Vitals, and LeakCanary when memory is involved. For security and release, assume the APK is inspectable and the client is not fully trusted: protect tokens, validate entry points, minimize exported surfaces, be careful with WebView bridges, and test minified release builds. R8, keep rules, mapping files, staged rollout, crash monitoring, and rollback strategy are part of the production answer, not afterthoughts."

Tricky follow-ups answered

Follow-up: What do you measure first?

Answer: Frame timing, main-thread blocking, startup phases, allocations, I/O, lock contention, crash rate, or security boundary depending on the issue.

Follow-up: What can release builds change?

Answer: R8 can remove or rename code used by reflection/serialization, change stack traces, and expose keep-rule gaps.

Follow-up: Can secrets be hidden in an APK?

Answer: No. The client is inspectable. Authorization must be enforced server-side and secrets should not rely on obscurity.

Follow-up: What is the production answer?

Answer: Use staged rollout, monitoring, mapping files, rollback/feature flags, and a small verified fix.

10. Architecture disagreement story.

Senior answer

"I would answer with one concrete story, but I would make the structure visible only through natural speech: context, constraint, action, trade-off, result, and what I changed afterward. For a senior Android role, the story should show impact beyond my own ticket: product alignment, technical judgment, risk management, communication, mentoring, and follow-through. I would avoid making other people the problem. If there was conflict, I would separate facts from preferences, show how I created options, and explain how the team converged. The answer should feel honest, specific, and reflective, not like a memorized leadership script."

Tricky follow-ups answered

Follow-up: What should the story prove?

Answer: Judgment, ownership, communication, learning, and impact beyond simply finishing a ticket.

Follow-up: What trade-off should you name?

Answer: Time, risk, scope, team adoption, migration cost, product impact, or reversibility.

Follow-up: How do you avoid sounding defensive?

Answer: Describe constraints, decisions, and learning. Avoid blaming personalities or making yourself the only reasonable person.

Follow-up: How do you show ownership without bragging?

Answer: Name the risk, the people affected, the trade-off you chose, the outcome, and what changed afterward.

Flashcards

> *Quality status: **Student practice draft**. Target: 100+ flashcards.*

Use these for fast recall. Answer before expanding the card.

Kotlin

- **Q:** `==` vs `===`? — **A:** `==` calls `equals` for structural equality. `===` checks reference identity.
- **Q:** What does data class `copy()` do? — **A:** It creates a new outer instance with selected properties changed. It is shallow.
- **Q:** When is `hashCode` used? — **A:** Hash-based collections such as `HashMap`, `HashSet`, and operations like `distinct`.
- **Q:** What is a platform type? — **A:** A Java-interoperability type where Kotlin does not know exact nullability.
- **Q:** Why can Kotlin still throw NPE? — **A:** `!!`, Java interop, platform types, bad initialization, `lateinit`, explicit throws, generic holes.
- **Q:** Sealed class vs enum? — **A:** Enum is fixed constants with same shape. Sealed hierarchy can carry different data per case.
- **Q:** Why does `reified` require `inline`? — **A:** JVM erases generic types; inline lets compiler substitute the actual type at call site.
- **Q:** `out` in generics? — **A:** Covariant producer. Safe to read/produce `T`.
- **Q:** `in` in generics? — **A:** Contravariant consumer. Safe to accept/consume `T`.
- **Q:** Extension functions dispatch? — **A:** They are statically resolved; they do not override class members.

Android Fundamentals

- **Q:** Does ViewModel survive rotation? — **A:** Yes, while scoped owner remains.
- **Q:** Does ViewModel survive process death? — **A:** No. It is recreated after process death.
- **Q:** What belongs in `SavedStateHandle`? — **A:** Small restoration state: IDs, filters, selected tab, simple form values.
- **Q:** What belongs in Room/DataStore? — **A:** Durable data, cached entities, auth/session data, preferences, pending work metadata.
- **Q:** Why avoid Activity reference in ViewModel? — **A:** ViewModel can outlive Activity instance and leak it.
- **Q:** Application context vs Activity context? — **A:** Application context lives for the process; Activity context is tied to UI lifecycle.

Coroutines And Flow

- **Q:** Are coroutines threads? — **A:** No. They run on threads but can suspend without blocking them.
- **Q:** Does `suspend` mean background? — **A:** No. It means the function can suspend; dispatcher/implementation determines thread.
- **Q:** `launch` returns? — **A:** `Job`.
- **Q:** `async` returns? — **A:** `Deferred<T>`.
- **Q:** What happens if `async` fails? — **A:** Exception is observed on `await`, while still participating in parent cancellation rules.

- **Q:** What is structured concurrency? — **A:** Coroutines are tied to parent scopes so lifetime, cancellation, and failures are controlled.
- **Q:** Why avoid swallowing `CancellationException`? — **A:** It can break cancellation and keep work running after scope cancellation.
- **Q:** Flow cold or hot? — **A:** Regular Flow is cold by default.
- **Q:** StateFlow use? — **A:** Hot state holder with current value, good for UI state.
- **Q:** SharedFlow use? — **A:** Hot shared stream with replay/buffer config, useful for shared emissions when lifecycle semantics fit.
- **Q:** `flowOn` affects? — **A:** Upstream operators before it.
- **Q:** `catch` catches? — **A:** Upstream exceptions from where it is placed.

Compose

- **Q:** What is recomposition? — **A:** Re-invoking composables whose observed state may have changed.
- **Q:** Does recomposition redraw everything? — **A:** No. Compose can recompose affected parts and skip unchanged work.
- **Q:** State hoisting? — **A:** Move state to the owner that needs to read/change it.
- **Q:** Does all state belong in ViewModel? — **A:** No. Ownership depends on lifetime and sharing.
- **Q:** `remember` vs `rememberSaveable`? — **A:** `remember` survives recomposition while in composition. `rememberSaveable` can survive config/process recreation for saveable values.
- **Q:** `LaunchedEffect` key controls? — **A:** When the effect cancels and restarts.
- **Q:** Mutable list not updating UI? — **A:** Compose may not observe in-place mutation; use observable state or immutable replacement.

Architecture

- **Q:** MVVM core idea? — **A:** UI renders state and sends events; ViewModel owns screen state; data/domain layers provide work/data.
- **Q:** Clean Architecture core idea? — **A:** Dependency direction and separation from framework details.
- **Q:** Repository responsibility? — **A:** Data boundary and policy: sources, mapping, caching, sync, errors.
- **Q:** UseCase when? — **A:** When it represents meaningful business operation or orchestration.
- **Q:** Single source of truth? — **A:** One authoritative place for current data/state.

System Design

- **Q:** Offline-first hardest part? — **A:** Offline writes, sync, conflicts, idempotency, user-visible state.
- **Q:** WorkManager use? — **A:** Deferrable persistent work with constraints/retry.
- **Q:** Foreground service use? — **A:** Immediate ongoing user-visible work.
- **Q:** Idempotency? — **A:** Repeating operation does not create unintended duplicate effects.
- **Q:** Cached reads vs offline writes? — **A:** Cached reads show stored data. Offline writes require persisted pending operations and sync policy.

Testing

- **Q:** Why use `runTest`? — **A:** Controls coroutine execution and virtual time.
- **Q:** Why inject dispatchers? — **A:** Avoid hardcoded dispatchers and make tests deterministic.
- **Q:** Fake vs mock? — **A:** Fake has behavior; mock verifies interactions.
- **Q:** StateFlow testing gotcha? — **A:** It has an initial value and may never complete.
- **Q:** Room migration test should verify? — **A:** Old data survives and new schema is valid.

Performance, Security, Release

- **Q:** ANR cause? — **A:** Main thread cannot respond in time, often blocking I/O, long work, locks, binder waits.
- **Q:** Jank diagnosis starts with? — **A:** Measurement/traces, not guessing.
- **Q:** Can APK secrets be hidden? — **A:** Not reliably. Client is not a trusted environment.
- **Q:** Certificate pinning trade-off? — **A:** Reduces some MITM risk but adds rotation/outage risk.
- **Q:** R8 can break? — **A:** Reflection, serialization, DI/framework entry points if keep rules are wrong.
- **Q:** Mapping files used for? — **A:** Deobfuscating release crash reports.

Soft Skills

- **Q:** Behavioral answer shape? — **A:** Context, constraint, action, trade-off, result, reflection.
- **Q:** Architecture disagreement should show? — **A:** Criteria, trade-offs, low ego, decision path.
- **Q:** Mistake story should show? — **A:** Ownership, correction, prevention, learning.
- **Q:** Mentorship story should show? — **A:** How the other developer became more independent.
- **Q:** `lateinit` risk? — **A:** Access before initialization throws `UninitializedPropertyAccessException`.
- **Q:** `lazy` initializes when? — **A:** On first access, using its configured thread-safety mode.
- **Q:** Value class use? — **A:** Type-safe lightweight wrapper, often for IDs or domain primitives.
- **Q:** Sealed UI state benefit? — **A:** Makes mutually exclusive states explicit and exhaustive.
- **Q:** Fragment view lifecycle matters because? — **A:** Fragment can outlive its View; bindings/collectors tied to View must be cleared/stopped.
- **Q:** PendingIntent is? — **A:** A token allowing another app/system component to perform an action as your app later.
- **Q:** Exported component risk? — **A:** Other apps can invoke it if exported; validate inputs and restrict exposure.
- **Q:** CPU-heavy dispatcher? — **A:** `Dispatchers.Default`.
- **Q:** Blocking I/O dispatcher? — **A:** Usually `Dispatchers.IO`.
- **Q:** `combine` vs `zip`? — **A:** `combine` emits when any source emits after all have values. `zip` pairs emissions.
- **Q:** `flatMapLatest` behavior? — **A:** Cancels previous inner flow when a new upstream value arrives.
- **Q:** `callbackFlow` needs? — **A:** `awaitClose` to unregister callbacks and clean up.
- **Q:** Compose stability helps? — **A:** Compose can skip recomposition when stable inputs are unchanged.
- **Q:** `derivedStateOf` use? — **A:** Cache derived state that changes less often than its inputs.
- **Q:** Lazy list keys help? — **A:** Preserve item identity across moves/updates.
- **Q:** Hilt scope controls? — **A:** Dependency lifetime.
- **Q:** Singleton scope should avoid? — **A:** Activity/View references and screen-specific state.

- **Q:** Qualify dispatchers why? — **A:** Avoid ambiguity and make test replacement easy.
- **Q:** Dependency inversion? — **A:** High-level policy depends on abstractions, not low-level details.
- **Q:** Offline write requires? — **A:** Persisted pending operation, retry policy, idempotency, conflict handling.
- **Q:** Token refresh concurrency issue? — **A:** Many 401s can trigger many refresh calls unless refresh is serialized/shared.
- **Q:** Logout with pending work? — **A:** Cancel, clear, or re-scope work based on product/security rules.
- **Q:** `MainDispatcherRule` purpose? — **A:** Replace Main dispatcher in coroutine tests.
- **Q:** `advanceUntilIdle()`? — **A:** Runs scheduled coroutine work until no tasks remain.
- **Q:** Turbine is for? — **A:** Testing Flow emissions.
- **Q:** UI test flakiness often comes from? — **A:** Timing, async work, animations, network/data dependencies, weak synchronization.
- **Q:** Cold start? — **A:** App process is not running and must be created.
- **Q:** Baseline profiles? — **A:** Precompile critical paths to improve startup/runtime performance.
- **Q:** Macrobenchmark? — **A:** Measures app-level performance such as startup and scrolling.
- **Q:** LeakCanary detects? — **A:** Retained objects that should have been garbage collected.
- **Q:** WebView bridge risk? — **A:** JavaScript bridge can expose native functionality to untrusted content.
- **Q:** Deep link security rule? — **A:** Treat link parameters as untrusted input.
- **Q:** R8 keep rule purpose? — **A:** Prevent shrinking/obfuscation from breaking reflection/framework-used code.
- **Q:** Mapping files? — **A:** Map obfuscated release stack traces back to original symbols.
- **Q:** Strong conflict story shows? — **A:** Decision criteria, listening, trade-offs, and outcome.
- **Q:** Strong mistake story shows? — **A:** Ownership, fix, prevention, learning.
- **Q:** Technical debt pitch should mention? — **A:** Delivery risk, regressions, cost, and options.
- **Q:** Leadership without authority shows? — **A:** Influence through clarity, trust, alignment, and follow-through.
- **Q:** Ambiguity answer should start with? — **A:** Clarifying requirements, constraints, and assumptions.
- **Q:** Senior answer should always include? — **A:** Ownership, trade-offs, and practical consequences.

References

References

> *Quality status: **76/100, Draft**. Target: **96+**. Main gap: group sources by chapter, annotate why each source matters, replace broad search links with concrete sources, and add last-verified dates.*

Last researched: 2026-05-25

Official Documentation

- Kotlin documentation (https://kotlinlang.org/docs/home.html)
- Kotlin data classes (https://kotlinlang.org/docs/data-classes.html)
- Kotlin equality (https://kotlinlang.org/docs/equality.html)
- Kotlin null safety (https://kotlinlang.org/docs/null-safety.html)
- Kotlin coroutines guide (https://kotlinlang.org/docs/coroutines-guide.html)
- Kotlin coroutine exception handling (https://kotlinlang.org/docs/exception-handling.html)
- Kotlin Flow documentation (https://kotlinlang.org/docs/flow.html)
- Kotlin generics (https://kotlinlang.org/docs/generics.html)
- Android app architecture (https://developer.android.com/topic/architecture)
- Android architecture recommendations (https://developer.android.com/topic/architecture/recommendations)
- Android background tasks overview (https://developer.android.com/develop/background-work/background-tasks)
- WorkManager reference (https://developer.android.com/reference/androidx/work/WorkManager.html)
- WorkManager persistent work docs (https://developer.android.com/topic/libraries/architecture/workmanager/)
- Kotlin flows on Android (https://developer.android.com/kotlin/flow)
- Jetpack Compose side effects (https://developer.android.com/develop/ui/compose/side-effects)
- Android offline-first architecture (https://developer.android.com/topic/architecture/data-layer/offline-first)
- Testing Kotlin coroutines on Android (https://developer.android.com/kotlin/coroutines/test)
- Android UI testing (https://developer.android.com/training/testing/ui-tests/behavior)
- Android memory guide (https://developer.android.com/topic/performance/memory)
- Android performance measurement (https://developer.android.com/topic/performance/measuring-performance)
- Hilt with Jetpack libraries (https://developer.android.com/training/dependency-injection/hilt-jetpack)
- Android security best practices (https://developer.android.com/privacy-and-security/security-best-practices)
- Android security checklist (https://developer.android.com/guide/topics/security/security)
- Android cryptography and Keystore (https://developer.android.com/privacy-and-security/cryptography)
- R8 app optimization docs (https://developer.android.com/build/shrink-code)

Android / Kotlin Interview Sources

- mohsenoid/Android-Interview-Questions (https://github.com/mohsenoid/Android-Interview-Questions)

- Android System Design FAQ (https://www.androidsystemdesign.dev/faq)
- InterviewBee Android Developer question bank (https://interviewbee.ai/resources/role-based-questions/android-developer)
- Droidly Android Interview Prep (https://droidly.io/)
- Dove Letter Android & Kotlin Interview Questions (https://doveletter.dev/interviews)
- Job Lobster Android Engineer Interview Questions 2026 (https://joblobster.ai/guides/interview-prep/android-engineer-interview-questions)
- Mobile Architecture mock interviews (https://mobilearchitecture.dev/)
- The Mobile Interview: Cracking the Mobile System Design Interview (https://themobileinterview.com/cracking-the-mobile-system-design-interview/)

Forum And Candidate Reports

- Reddit: recent Android interviews (https://www.reddit.com/r/androiddev/comments/1dw19ub/those_of_you_who_have_given_android_interviews/)
- Reddit: Android dev returning after gap, skill gaps and recent interview themes (https://www.reddit.com/r/androiddev/comments/1qaxv8e/android_dev_8_yoe_returning_after_gap_need_blunt/)
- Reddit: Senior Android interview checklist (https://www.reddit.com/r/androiddev/comments/13yv1tt)
- Reddit: Android concurrency interview review (https://www.reddit.com/r/androiddev/comments/1tcgqsz/amo_concurrency_android_interview_review/)
- Reddit: common coroutine mistakes in Android (https://www.reddit.com/r/androiddev/comments/1scf1mt/what_are_common_mistakes_when_using_coroutines_in/)
- Reddit: coroutines, threads, and RxJava (https://www.reddit.com/r/androiddev/comments/1ldv0vv)
- Reddit: Senior interview discussion (https://www.reddit.com/r/androiddev/comments/1r8nkex/senior_interview/)
- Reddit: experienced Android developer knowledge discussion (https://www.reddit.com/r/androiddev/comments/1r2zy9w/what_should_an_experienced_android_developer/)
- Reddit: Compose UI state questions (https://www.reddit.com/r/androiddev/comments/1nnqe51)
- Reddit: Compose interview questions (https://www.reddit.com/r/androiddev/comments/1glbg3e)
- Reddit: Compose stability and skipping discussion (https://www.reddit.com/r/androiddev/comments/1svzn7d/strong_skipping_mode_in_jetpack_compose_common/)
- Reddit: LazyColumn recomposition and stable list discussion (https://www.reddit.com/r/androiddev/comments/15vc37e/recomposition_of_all_items_using_listinterface/)
- Reddit: Clean Architecture overkill (https://www.reddit.com/r/androiddev/comments/1eppoaf)
- Reddit: MVVM and Clean Architecture in Compose (https://www.reddit.com/r/androiddev/comments/1rh3542/struggling_to_understand_mvvm_clean_architecture/)
- Reddit: Android system design interviews (https://www.reddit.com/r/androiddev/comments/mrxgkr)
- Reddit: Android foreground services after Android 12 (https://www.reddit.com/r/androiddev/comments/1cvd3ia)
- Reddit: Android security discussions (https://www.reddit.com/r/androiddev/search/?q=security%20interview)
- Reddit: Android modularization discussions (https://www.reddit.com/r/androiddev/search/?q=modularization%20interview)

- Reddit: Google L5 Android system design (https://www.reddit.com/r/androiddev/comments/1e0kjtu/interviewing_with_google_for_an_l5_role_android/)
- Reddit: Google Android interview (https://www.reddit.com/r/androiddev/comments/1mchwbe/android_developer_google_interview/)
- Reddit: Meta Android E6 system design (https://www.reddit.com/r/interviewpreparations/comments/1mw4c1b)
- Reddit: Meta Android interview experience (https://www.reddit.com/r/leetcode/comments/1nagyra/meta_software_engineer_android_interview/)
- Glassdoor: Senior Android Developer interview questions (https://www.glassdoor.com/Interview/senior-android-developer-interview-questions-SRCH_KO0%2C24_SDRD.htm)
- Glassdoor: Airbnb Senior Android Engineer interview report (https://www.glassdoor.com/Interview/Airbnb-Interview-RVW95616026.htm)

Big-Company And Senior Interview Guides

- Prepfully Meta Android Engineer Guide (https://prepfully.com/interview-guides/meta-android-engineer)
- Dataford Meta Mobile Engineer Guide (https://dataford.io/interview-guides/meta/mobile-engineer)
- IGotAnOffer Meta system design interview (https://igotanoffer.com/blogs/tech/meta-system-design-interview)
- TechInterview Airbnb interview guide (https://www.techinterview.org/airbnb-interview-guide/)
- TechInterview OOP design patterns interview guide (https://www.techinterview.org/post/3233474138/coding-interview-oop-design-patterns-strategy-observer-factory-singleton-decorator-adapter-solid-principles/)
- Algoroq design patterns for senior engineers (https://www.algoroq.io/interview-questions/design-patterns/)

Soft Skills / Behavioral Sources

- ExperiencedDevs: behavioral interview discussion (https://www.reddit.com/r/ExperiencedDevs/comments/t6qiwq)
- ExperiencedDevs: applying for senior, behavioral focus (https://www.reddit.com/r/ExperiencedDevs/comments/1ekv4ed)
- ExperiencedDevs: how to interview senior engineers (https://www.reddit.com/r/ExperiencedDevs/comments/1ssoua3/how_do_you_interview_senior_software_engineer/)
- Teal: behavioral interview questions for software engineering managers (https://www.tealhq.com/career-paths/software-engineering-manager-interview-questions/)
- EM Tools: conflict resolution interview questions (https://www.em-tools.io/interview-questions/conflict-resolution)
- DataAnnotation: senior software engineer behavioral questions (https://www.dataannotation.tech/developers/senior-software-engineer-behavioral-interview-questions)